
LeakGFN: Robust Molecular Generation in Generative Flow Networks via Flow Decomposition

Hwanhee Kim¹ Seungyeon Choi¹ Sanghyun Park¹

Abstract

Generative Flow Networks (GFlowNets) sample diverse molecules proportionally to a reward, but the vast chemical space forces trajectory truncation, so incomplete fragments become terminal states alongside valid molecules. We formalize the resulting distortion as *flow leakage*, the probability mass a converged GFlowNet allocates to these forced terminals. We propose **LEAKGFN**, which decomposes flow into a *chemical head* that keeps add-action flow alive at the truncation boundary and a *valid head* that estimates the fraction of that flow reaching valid molecules from flow matching alone. The decomposition acts on *training dynamics* near the boundary rather than merely relabeling the target. Separate gradient pathways for flow magnitude and completion decouple the conflicting signals that destabilize single-head models, and our ablations attribute the dominant gain to it rather than to exploration. We also express the residual leakage in closed form and show that at a reward-proportional fixed point the sampler is exact over accessible molecules once conditioned on completion. Experiments on five molecular optimization tasks show the best or tied-best HM on four of them, and the module plugs into existing frameworks, improving pocket-conditioned and multi-objective generation. Code is available at <https://github.com/HwanheeKim813/LeakGFN>.

1. Introduction

Discovering novel molecules with desired properties is a fundamental challenge in drug discovery and material design (Hains et al., 2010; Hughes et al., 2011). While tradi-

¹Department of Computer Science, Yonsei University, Seoul, Korea. Correspondence to: Sanghyun Park <sanghyun@yonsei.ac.kr>.

tional virtual screening evaluates molecules from predefined libraries, *de novo* molecular generation methods explore a vast chemical space, estimated to contain at least 10^{23} drug-like molecules (Polishchuk et al., 2013), by generating structures from scratch. Recent deep generative models, including variational autoencoders (Gómez-Bombarelli et al., 2018; Jin et al., 2018), reinforcement learning (Olivecrona et al., 2017), and Generative Flow Networks (GFLOWNETS) (Bengio et al., 2021; 2023), have shown remarkable promise in this domain. GFLOWNETS are particularly attractive for molecular generation as they sample diverse candidates with probability proportional to a reward function (Bengio et al., 2021). Unlike reinforcement learning methods that converge to single optima, GFLOWNETS naturally explore multiple high-reward modes. This diversity is crucial for drug discovery, where multiple candidates are essential for downstream optimization (Jain et al., 2023; Zhu et al., 2023).

While GFLOWNETS effectively address the mode collapse problem in well-defined finite spaces, they face fundamental challenges when applied to domains with immense state spaces, such as chemical space. To make computation tractable in such vast spaces, existing works typically truncate the trajectory length, thereby constraining the reachable state space to a manageable size. Although this truncation is necessary for computational efficiency, it introduces a critical *target distribution mismatch*. While flow conservation still holds within the truncated DAG, the model learns a distribution over the incorrect target space that includes both valid molecules and incomplete fragments. As illustrated in Figure 1(A), conventional approaches treat states at the truncation boundary as *forced terminals*, assigning them probability mass alongside valid molecules. This conflates two fundamentally different outcomes, namely complete molecules with meaningful rewards and incomplete fragments that happen to reach the length limit. Consequently, the learned distribution allocates probability to incomplete outputs rather than finished molecules, reducing the effective capacity for exploring valid chemical space.

Prior works have addressed related challenges through improved training objectives (Madan et al., 2023; Pan et al., 2023), exploration strategies (Kim et al., 2024b;a), and active learning (Jain et al., 2022). However, none directly

tackle the fundamental issue of learning over an incorrect target space induced by truncation. This problem becomes increasingly severe for larger molecules, which is precisely the regime relevant for complex drug scaffolds.

We propose LEAKGFN, which shifts probability mass toward valid molecules by explicitly decomposing flow into two components (Figure 1(B)). The *chemical flow* F_{chem} is a base flow whose add-action outflow at the truncation boundary is kept positive rather than driven to zero, while the *valid flow* F_{valid} represents flow toward molecules that complete within the boundary. Our architecture employs a *chemical head* that models flow magnitude over all transitions, and a *valid head* that estimates the fraction of flow reaching accessible terminals. Through the decomposition $F_{\text{valid}} = F_{\text{chem}} \cdot \lambda$, the valid head implicitly learns a reachability-related gate without explicit labels. This lets the sampler allocate its budget toward completions that fit within the boundary. At a reward-proportional fixed point, conditioning on valid completion recovers the target distribution over accessible molecules, and we give a closed-form expression for the residual mass left on incomplete fragments. As Remark 3.3 makes precise, that conditional property is shared by any converged reward-proportional GFlowNet, and the empirical benefit of the decomposition lies in finite-training stability under truncation.

Crucially, our dual-head decomposition improves *training dynamics* near the truncation boundary rather than merely changing the target distribution. By providing the model with separate gradient pathways for chemical flow and reachability estimation, the architecture resolves conflicting learning signals that arise when a single flow function must simultaneously model flow toward valid terminals and flow toward the boundary. Our ablation studies confirm that this training dynamics improvement, not the exploration strategy, accounts for the dominant performance gain.

Our main contributions are as follows:

- **Problem Identification and Formalization.** We identify that trajectory truncation causes GFlowNets to learn over an incorrect target space that conflates valid molecules with incomplete fragments, and provide the first formal definition of this *flow leakage* phenomenon (Definition 3.1).
- **LeakGFN: A Principled Solution.** We propose a dual-head architecture that decomposes flow into chemical and valid components, improving training dynamics by decoupling gradient signals for flow magnitude and reachability. The valid head implicitly learns molecular reachability through asymmetric terminal treatment, requiring no explicit supervision. We prove that, conditioned on producing a complete molecule, the sampler matches the target distribution over accessible

molecules, and we derive a closed-form expression for the residual leakage.

- **Comprehensive Empirical Validation.** We demonstrate the best or tied-best HM on four of five molecular optimization benchmarks, with notable robustness to hyperparameter selection (L_{max}). Ablation studies confirm the decomposition accounts for the dominant improvement independent of exploration strategy. Our plug-and-play module improves existing frameworks on pocket-conditioned and multi-objective generation tasks.

2. Related Work

Generative Flow Networks. GFlowNets (Bengio et al., 2021) sample compositional objects proportionally to a reward function. Proposed training objectives include flow matching (FM) (Bengio et al., 2021), trajectory balance (TB) (Malkin et al., 2022), detailed balance (Bengio et al., 2023), and subtrajectory balance (Madan et al., 2023). Our method is orthogonal to these objectives. For molecular generation, both atom-level (Bengio et al., 2021) and fragment-based (Jin et al., 2020; Bengio et al., 2023) approaches have been explored, with recent extensions to pocket-conditioned generation (Shen et al., 2024; Jiang et al., 2024). Our dual-head architecture integrates seamlessly into these frameworks.

Exploration and Large State Spaces. Effective exploration strategies include ϵ -greedy, Thompson sampling (Rector-Brooks et al., 2023), local search (Kim et al., 2024b), and genetic operators (Kim et al., 2024a). For large state spaces, Pan et al. (2023) proposed forward-looking objectives. However, none of these methods address the fundamental issue that truncation causes learning over an incorrect target space. To our knowledge, LEAKGFN is the first to explicitly model flow leakage and to characterize, in closed form, the residual probability mass it leaves on incomplete fragments.

3. Method

We first review GFlowNets and formally characterize the truncation problem, then present LEAKGFN.

3.1. Preliminaries: GFlowNets

A GFlowNet (Bengio et al., 2021) samples compositional objects $x \in \mathcal{X}$ with probability proportional to a reward $R(x) \geq 0$. Objects are built by a sequence of actions in a directed acyclic graph $\mathcal{G} = (\mathcal{S}, \mathcal{A})$, where each trajectory $\tau = (s_0 \rightarrow \dots \rightarrow s_n)$ starts at s_0 and ends at a terminal state $s_n \in \mathcal{X}$.

GFlowNets define a *flow function* $F : \mathcal{A} \rightarrow \mathbb{R}_{\geq 0}$ over edges,

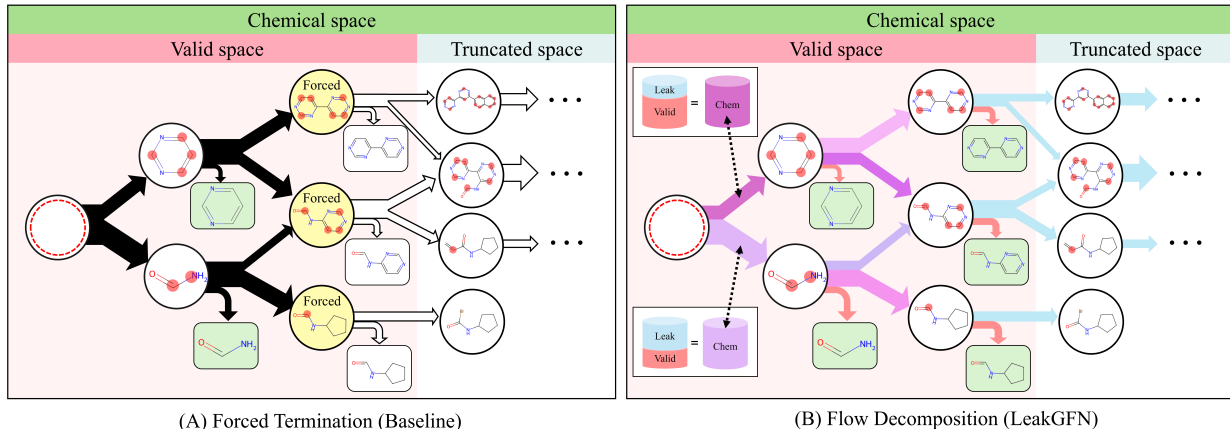


Figure 1. Comparison of state space formulations under trajectory truncation. **(A) Forced Termination:** Conventional GFlowNets treat states at the truncation boundary as forced terminals (yellow), assigning them probability mass alongside valid terminals (green). Flow beyond the boundary (white) is ignored, resulting in a distribution over both valid molecules and incomplete structures. **(B) Flow Decomposition (LeakGFN):** Our method decomposes flow into valid (red) and leak (blue) components. The chemical flow (purple) at each state is a mixture that gradually separates as trajectories progress. The valid flow defines the sampling policy, while the complementary leak component represents continuation beyond the truncation boundary. Residual probability can still terminate at forced states, as characterized in Theorem 3.2.

where $F(s \rightarrow s')$ denotes the flow along the edge from state s to state s' . The key principle is *flow conservation*, which requires that for any non-terminal state $s \neq s_0$, the incoming flow equals the outgoing flow:

$$\sum_{s' \in \text{Pa}(s)} F(s' \rightarrow s) = \sum_{s'' \in \text{Ch}(s)} F(s \rightarrow s'') \quad (1)$$

where $\text{Pa}(s)$ and $\text{Ch}(s)$ denote parents and children of s , respectively. The initial state has no parent, and its outgoing flow defines the total flow $F^{\text{out}}(s_0) = Z$. For terminal states $x \in \mathcal{X}$, the incoming flow equals the reward:

$$\sum_{s' \in \text{Pa}(x)} F(s' \rightarrow x) = R(x) \quad (2)$$

When these conditions are satisfied, sampling trajectories by following the forward policy $\pi(s'|s) \propto F(s \rightarrow s')$ yields terminal states with the desired distribution:

$$P(x) = \frac{R(x)}{\sum_{x' \in \mathcal{X}} R(x')} \propto R(x) \quad (3)$$

3.2. Problem: Truncation Induces Incorrect Target Space

In molecular generation, the state space is enormous. To ensure computational tractability, existing methods impose a maximum trajectory length $L_{\max}$, truncating generation at a fixed depth (Bengio et al., 2023). This creates two disjoint types of terminal states ($\mathcal{T}_{\text{valid}} \cap \mathcal{T}_{\text{forced}} = \emptyset$). *Valid terminals* $\mathcal{T}_{\text{valid}}$ are states at which generation ends with a *complete*

molecule, either because the agent chose the stop action or because no attachment points remain. *Forced terminals* $\mathcal{T}_{\text{forced}}$ are states that reach $L_{\max}$ while attachment points are still open. Here termination is imposed by the truncation budget rather than by completion of the molecule.

Conventional GFlowNets treat both as terminals, learning a distribution over $\mathcal{T}_{\text{valid}} \cup \mathcal{T}_{\text{forced}}$. However, this is fundamentally incorrect. Forced terminals are incomplete fragments, not valid molecules. We formally characterize this phenomenon:

Definition 3.1 (Flow Leakage). Given a GFlowNet with truncation boundary $L_{\max}$, *flow leakage* is the probability mass allocated to forced terminal states $\mathcal{T}_{\text{forced}}$ under the learned distribution:

$$\mathcal{L}_{\text{leak}} = \sum_{s \in \mathcal{T}_{\text{forced}}} P(s) = \frac{\sum_{s \in \mathcal{T}_{\text{forced}}} R(s)}{\sum_{s \in \mathcal{T}_{\text{forced}}} R(s) + \sum_{x \in \mathcal{T}_{\text{acc}}} R(x)} \quad (4)$$

where $P(s)$ is the probability assigned to state s under the converged GFlowNet policy. Flow leakage directly reduces the effective probability mass available for sampling valid molecules, distorting the learned distribution away from the target.

The correct target is the distribution over *accessible* valid molecules:

$$P_{\text{target}}(x) = \frac{R(x)}{\sum_{x' \in \mathcal{T}_{\text{acc}}} R(x')} \quad \text{for } x \in \mathcal{T}_{\text{acc}} \quad (5)$$

where $\mathcal{T}_{\text{acc}} \subseteq \mathcal{T}_{\text{valid}}$ denotes the set of valid molecules reachable within $L_{\max}$ steps (in our setting every valid terminal

is reachable by construction, so $\mathcal{T}_{\text{acc}} = \mathcal{T}_{\text{valid}}$), $R(x)$ is the reward for molecule x , and the denominator normalizes over all accessible molecules.

3.3. LeakGFN: Flow Decomposition

Our key insight is that flow at any state can be decomposed into two components based on reachability within the truncation boundary.

Dual-Head Architecture. We propose a dual-head architecture that shares a common state-action encoder θ_{emb} but uses separate heads for predicting flow magnitude and reachability.

The *chemical head* θ_{chem} models flow without the artificial truncation constraint, including the continuation just beyond the boundary:

$$F_{\text{chem}}(s, a) = \exp(\theta_{\text{chem}}(\theta_{\text{emb}}(s, a))) \quad (6)$$

where $\theta_{\text{emb}}(s, a)$ is a shared embedding of the state-action pair, and the exponential ensures non-negative flow. This head has the same architecture as standard GFlowNet flow predictors, outputting log-scale flow values.

The *valid head* θ_{valid} estimates the fraction of flow that will reach accessible terminal states within the truncation boundary:

$$\lambda(s, a) = \sigma(\theta_{\text{valid}}(\theta_{\text{emb}}(s, a))) \quad (7)$$

where $\sigma(\cdot)$ denotes the sigmoid function, ensuring $\lambda(s, a) \in [0, 1]$. Intuitively, $\lambda(s, a)$ represents the fraction of flow through action a at state s that will terminate at a valid molecule rather than being truncated.

The *valid flow*, which defines the sampling policy, combines these components for add actions:

$$F_{\text{valid}}(s, a) = F_{\text{chem}}(s, a) \cdot \lambda(s, a) \quad \text{for } a \in \mathcal{A}_{\text{add}} \quad (8)$$

where $\mathcal{A}_{\text{add}}$ denotes the set of fragment addition actions. The complementary *leak flow* $F_{\text{leak}}(s, a) = F_{\text{chem}}(s, a) \cdot (1 - \lambda(s, a))$ represents flow that would exit through the truncation boundary. For the stop action we use a single shared head $F(s, \text{stop})$ without decomposition. Stopping terminates generation at the current state with reward $R(s)$, so no reachability estimate is needed. As illustrated in Figure 1(B), the chemical flow (purple) at early states is a mixture that gradually separates into valid flow (red) toward accessible molecules and leak flow (blue) toward the boundary.

3.4. Training Objective

The cornerstone of LEAKGFN’s implicit reachability learning is the asymmetric treatment of forced terminals between the two heads, which enables λ to capture molecular accessibility without explicit supervision.

Asymmetric Terminal Treatment. We train the two heads with different treatments of terminal states, summarized in Table 1.

Table 1. Losses applied at each terminal type. “Term.” is the terminal loss and “Cons.” the flow-conservation loss.

	F_{chem}		F_{valid}	
	Term.	Cons.	Term.	Cons.
Valid terminal	–	–	✓	–
Forced terminal	–	✓	✓	✓

For valid terminals $s \in \mathcal{T}_{\text{valid}}$, only the valid head is trained, and the stop edge carries the reward:

$$F(s, \text{stop}) = R(s) \quad (9)$$

For forced terminals $s \in \mathcal{T}_{\text{forced}}$, the treatments differ. The valid head treats them as terminals with their oracle rewards:

$$F_{\text{valid}}^{\text{in}}(s) = R(s) \quad (10)$$

where $R(s)$ is the oracle evaluation of the incomplete fragment. In fragment-based generation, every state corresponds to a valid molecular graph that the oracle can evaluate. Forced terminals typically receive low rewards because incomplete fragments with open attachment points lack the structural completeness required for high bioactivity or drug-likeness. The chemical head treats them as non-terminals, enforcing flow conservation:

$$F_{\text{chem}}^{\text{in}}(s) = F_{\text{chem}}^{\text{out}}(s) = F(s, \text{stop}) + \sum_{a \in \mathcal{A}_{\text{add}}} F_{\text{chem}}(s, a) \quad (11)$$

This allows F_{chem} to model outgoing flow through add actions as if the trajectory could continue beyond the truncation boundary.

Implicit Learning of λ . At a forced terminal s , F_{valid} incurs both the terminal and the flow conservation loss:

$$F_{\text{valid}}^{\text{in}}(s) = R(s) \quad (\text{terminal condition}) \quad (12)$$

$$F_{\text{valid}}^{\text{in}}(s) = F_{\text{valid}}^{\text{out}}(s) \quad (\text{flow conservation}) \quad (13)$$

Thus $F_{\text{valid}}^{\text{out}}(s) = F(s, \text{stop}) + \sum_a F_{\text{valid}}(s, a) = R(s)$.

Ensuring $F(s, \text{stop}) = R(s)$ at terminals requires explicit supervision, since the stop logit is otherwise unconstrained. We therefore add an auxiliary transition $s \xrightarrow{\text{stop}} z_s$ into a dedicated absorbing terminal z_s for every terminal state (both valid and forced), treated as a valid terminal with reward $R(s)$. The state z_s shares the molecular features of s but carries a distinct terminal identity and has no outgoing actions, so the state graph remains acyclic. This explicitly trains $F(s, \text{stop}) = R(s)$ via the terminal loss. Two

constraints are therefore in play and should not be conflated. The terminal loss of Eq. 16 acts on the incoming flow $F_{\text{valid}}^{\text{in}}(s)$, whereas the auxiliary transition acts on the stop edge $F(s, \text{stop})$, which comes from the shared stop head and so from neither column of Table 1. Both target $R(s)$, and at a forced terminal they combine with flow conservation to drive:

$$\sum_{a \in \mathcal{A}_{\text{add}}} F_{\text{valid}}(s, a) \rightarrow 0 \quad (14)$$

Meanwhile, F_{chem} computes only flow conservation loss at forced terminals, maintaining positive flow through add actions ($F_{\text{chem}}(s, a) > 0$ for $a \in \mathcal{A}_{\text{add}}$). Consequently,

$$\lambda(s, a) = \frac{F_{\text{valid}}(s, a)}{F_{\text{chem}}(s, a)} \rightarrow 0 \quad \text{for all } a \in \mathcal{A}_{\text{add}} \quad (15)$$

Because λ is a clamped sigmoid (Appendix A), these are limits approached during optimization rather than exact identities.

This signal propagates backward through the DAG via the flow matching objective. $\lambda(s, a)$ stays close to 1 along trajectories that complete within the boundary and falls toward 0 on actions leading to it, so λ learns reachability without explicit supervision.

Flow Matching Loss. We train both heads using the flow matching objective. For F_{valid} , we decompose the loss into terminal and flow conservation components:

$$\mathcal{L}_{\text{valid}}^{\text{term}} = \mathbb{E}_{s \in \mathcal{T}_{\text{valid}} \cup \mathcal{T}_{\text{forced}}} \left[\left(\log \frac{F_{\text{valid}}^{\text{in}}(s)}{R(s)} \right)^2 \right] \quad (16)$$

$$\mathcal{L}_{\text{valid}}^{\text{flow}} = \mathbb{E}_{s \notin \mathcal{T}_{\text{valid}} \cup \{s_0\}} \left[\left(\log \frac{F_{\text{valid}}^{\text{in}}(s)}{F_{\text{valid}}^{\text{out}}(s)} \right)^2 \right] \quad (17)$$

where each expectation is a mean over the sampled states of the corresponding type. The terminal loss therefore covers both kinds of terminal, while the conservation loss covers non-terminals and forced terminals.

For F_{chem} , we apply only the flow conservation loss at non-valid-terminal states:

$$\mathcal{L}_{\text{chem}} = \mathbb{E}_{s \notin \mathcal{T}_{\text{valid}} \cup \{s_0\}} \left[\left(\log \frac{F_{\text{chem}}^{\text{in}}(s)}{F_{\text{chem}}^{\text{out}}(s)} \right)^2 \right] \quad (18)$$

The total loss is $\mathcal{L} = \mathcal{L}_{\text{valid}} + \alpha \mathcal{L}_{\text{chem}}$, where $\mathcal{L}_{\text{valid}} = \gamma \mathcal{L}_{\text{valid}}^{\text{term}} + \mathcal{L}_{\text{valid}}^{\text{flow}}$ with terminal loss coefficient γ , and $\alpha \geq 0$ controls the strength of chemical flow regularization. A larger α encourages the model to learn flow dynamics that extend beyond forced terminals, improving the estimation of leak flow at states near the truncation boundary. When $\alpha = 0$, the chemical head receives no direct supervision. It is still trained indirectly through the coupling

$F_{\text{valid}} = F_{\text{chem}} \cdot \lambda$, as gradients from $\mathcal{L}_{\text{valid}}$ backpropagate through both heads. We use $\alpha = 1$ in all experiments unless otherwise specified. We provide sensitivity analysis on α in Appendix C.

3.5. Flow-Guided Exploration

Standard GFlowNet training uses ϵ -exploration, mixing the learned policy with uniform random actions:

$$\pi_{\text{explore}}(a|s) = (1 - \epsilon) \cdot \pi(a|s) + \epsilon \cdot \text{Uniform}(a) \quad (19)$$

where $\pi(a|s)$ is the learned policy, $\epsilon \in [0, 1]$ controls the exploration rate, and $\text{Uniform}(a)$ assigns equal probability to all valid actions.

Under truncation, this strategy is inefficient. Uniform sampling wastes probability on actions leading toward the truncation boundary, generating trajectories that end at forced terminals, whose terminal signal is uninformative about complete molecules.

We propose an alternative in which the exploration branch of the behavior policy π_b draws from the *chemical* flow instead of from a uniform distribution:

$$\pi_b(a|s) = (1 - \epsilon) \pi_{\text{valid}}(a|s) + \epsilon \pi_{\text{chem}}(a|s), \quad (20)$$

where π_{valid} and π_{chem} normalize $\{F(s, \text{stop})\} \cup \{F_{\text{valid}}(s, a)\}_{a \in \mathcal{A}_{\text{add}}}$ and $\{F(s, \text{stop})\} \cup \{F_{\text{chem}}(s, a)\}_{a \in \mathcal{A}_{\text{add}}}$ respectively. With probability ϵ the action is drawn from the chemical-flow policy rather than from a uniform perturbation, which also shifts the stop probability because the two policies use different normalizers. The stop logit is shared, and is masked for the first L_{min} steps. We use $\epsilon = 0.05$ (Table 8).

Comparison with ϵ -exploration. Standard ϵ -exploration perturbs the policy with a uniform distribution that ignores chemistry entirely. Our flow-guided strategy (Eq. 20) instead substitutes a chemically informed proposal. F_{chem} is trained on the same transitions as F_{valid} and therefore concentrates on plausible fragment additions, so exploratory steps stay on chemically plausible transitions without explicit action masking.

We use F_{chem} rather than F_{valid} for exploration because F_{valid} diminishes near the truncation boundary as probability mass shifts toward forced terminals. In contrast, F_{chem} remains well-defined throughout the chemical space, providing stable gradients and guiding exploration toward chemically plausible regions.

3.6. Learned Distribution and Residual Leakage

We now characterize the distribution that LEAKGFN samples from at convergence, together with the probability mass that still leaks to forced terminals.

Theorem 3.2 (Learned distribution and residual leakage). *Suppose the flow matching conditions of Section 3.4 hold for F_{valid} at convergence, namely (i) flow conservation at every non-valid-terminal state and (ii) incoming flow equals reward at every terminal. Formally,*

$$F_{\text{valid}}^{\text{in}}(s) = F_{\text{valid}}^{\text{out}}(s) \quad \forall s \notin \mathcal{T}_{\text{valid}} \cup \{s_0\} \quad (21)$$

$$F_{\text{valid}}^{\text{in}}(z) = R(z) \quad \forall z \in \mathcal{T}_{\text{acc}} \cup \mathcal{T}_{\text{forced}} \quad (22)$$

Then sampling trajectories by following the forward policy $\pi(a|s) \propto F_{\text{valid}}(s, a)$ yields

$$P(z) = \frac{R(z)}{Z_{\text{valid}}}, \quad Z_{\text{valid}} = \sum_{x \in \mathcal{T}_{\text{acc}}} R(x) + \sum_{s \in \mathcal{T}_{\text{forced}}} R(s), \quad (23)$$

and the residual flow leakage of Definition 3.1 is

$$\mathcal{L}_{\text{leak}} = \frac{\sum_{s \in \mathcal{T}_{\text{forced}}} R(s)}{Z_{\text{valid}}}. \quad (24)$$

Conditioned on terminating at an accessible molecule, the sampler recovers the target distribution exactly:

$$P(x \mid x \in \mathcal{T}_{\text{acc}}) = \frac{R(x)}{\sum_{x' \in \mathcal{T}_{\text{acc}}} R(x')} = P_{\text{target}}(x). \quad (25)$$

Proof. By condition (i) all flow leaving s_0 is eventually absorbed at a terminal, and by condition (ii) the amount absorbed at terminal z is $R(z)$. Summing over both terminal types gives Z_{valid} . A terminal is then sampled with probability equal to its share of that total, $P(z) = F_{\text{valid}}^{\text{in}}(z)/Z_{\text{valid}} = R(z)/Z_{\text{valid}}$. Equation 24 follows by summing over $\mathcal{T}_{\text{forced}}$, and Eq. 25 by renormalizing over $\mathcal{T}_{\text{acc}}$. $\square$

Both conditions hold only in the limit. Together they force $\sum_{a \in \mathcal{A}_{\text{add}}} F_{\text{valid}}(s, a) = 0$ at forced terminals, whereas λ is clamped below at 10^{-20} and $F_{\text{chem}}(s, a) > 0$, so the sum stays strictly positive and the residual add-flow leaves an error in the normalization of Eq. 23 that scales with $\max_{s \in \mathcal{T}_{\text{forced}}, a \in \mathcal{A}_{\text{add}}} \lambda(s, a)$. Equation 24 coincides with Definition 3.1, so at this fixed point LEAKGFN leaks the same mass as a standard GFlowNet over the same terminal set. Its practical content is Eq. 25. Whenever LEAKGFN terminates at a complete molecule, that molecule is drawn exactly from the target distribution of Eq. 5, and the only distortion is the residual mass spent on forced terminals, which scales with how many of them exist as well as with their rewards. **Remark 3.3.** At forced terminals the asymmetric treatment acts on the *outgoing* flow rather than the incoming flow. The terminal loss of Eq. 16 applies at these states, so at convergence $F_{\text{valid}}^{\text{in}}(s) = R(s) > 0$, while flow conservation drives the add-flow to vanish, $\sum_{a \in \mathcal{A}_{\text{add}}} F_{\text{valid}}(s, a) = 0$. Reward is still absorbed at the boundary, and what the decomposition removes is the flow spent past it. A zero residual

would additionally require $R(s) = 0$ at forced terminals, which Section 4.3 shows destabilizes training, and λ in Appendix D behaves accordingly, decreasing with depth but plateauing above zero. Equation 25 describes a conditional fixed point shared by any converged GFlowNet that is reward-proportional over $\mathcal{T}_{\text{acc}} \cup \mathcal{T}_{\text{forced}}$, once restricted to $\mathcal{T}_{\text{acc}}$. LEAKGFN’s contribution is therefore a matter of finite-training dynamics under truncation, shown empirically in Section 4, rather than a different fixed point. Like prior GFlowNet analyses (Bengio et al., 2021; Malkin et al., 2022), we establish properties at convergence and leave formal convergence guarantees open for flow-based models.

4. Experiments

We evaluate LEAKGFN on a diverse set of molecular optimization tasks, comparing against state-of-the-art baselines and conducting comprehensive ablation studies.

4.1. Experimental Setup

Tasks. We evaluate on five optimization tasks spanning diverse molecular properties. Three tasks target protein bioactivity, namely the kinases JNK3 and GSK3 β and the dopamine D2 receptor (DRD2). For each of these, an activity score is produced by a classifier trained on bioactivity data, using random forests for JNK3 and GSK3 β and a support vector machine for DRD2 (Appendix A). The remaining two tasks target molecular properties, QED (quantitative estimate of drug-likeness) and SA (synthetic accessibility).

Baselines. We compare against several categories of methods: (1) genetic algorithm baselines (Graph-GA) (Jensen, 2019); (2) reinforcement learning methods including REINVENT (Blaschke et al., 2020) and GEGL (Ahn et al., 2020); and (3) GFlowNet variants including the standard flow matching objective (GFN-FM) (Bengio et al., 2021), trajectory balance (GFN-TB) (Malkin et al., 2022), subtrajectory balance (GFN-SubTB) (Madan et al., 2023) and detailed balance (GFN-DB) (Bengio et al., 2023).

Metrics. We evaluate methods using three complementary metrics with $k = 100$: (1) *Score*, the mean oracle score of the top- k molecules; (2) *Diversity* (Div), the average pairwise Tanimoto dissimilarity (on Morgan fingerprints) among top- k molecules; and (3) *Uniqueness* (Uni), the fraction of unique molecules among all 1,000 generated samples. We report the harmonic mean (HM) of these three metrics as the primary evaluation criterion. HM is computed within each seed and then averaged across seeds. Unlike arithmetic mean, HM is sensitive to low values, so a method must perform well on all three rather than excelling in one. All experiments are conducted with 3 random seeds, and we report mean $\pm$ standard deviation.

Table 2. Comparison of molecular optimization methods on the diverse molecular properties. We report HM score ($\uparrow$) averaged over 3 random seeds. Best results are in **bold**, second-best results are underlined. Several gaps are smaller than the seed standard deviation and should not be read as significant.

	JNK3	GSK3 β	DRD2	QED	SA
Graph-GA	0.521 $\pm$ 0.009	0.558 $\pm$ 0.006	0.667 $\pm$ 0.029	0.709 $\pm$ 0.008	0.625 $\pm$ 0.008
REINVENT	0.560 $\pm$ 0.071	0.612 $\pm$ 0.029	0.534 $\pm$ 0.059	0.692 $\pm$ 0.104	0.540 $\pm$ 0.006
GEGL	0.593 $\pm$ 0.039	0.625 $\pm$ 0.142	0.710 $\pm$ 0.071	0.869 $\pm$ 0.009	0.674 $\pm$ 0.009
GFN-DB	0.623 $\pm$ 0.003	0.673 $\pm$ 0.001	0.934 $\pm$ 0.002	0.897 $\pm$ 0.044	0.864 $\pm$ 0.007
GFN-TB	<u>0.624 $\pm$ 0.003</u>	0.675 $\pm$ 0.008	<u>0.933 $\pm$ 0.000</u>	<u>0.899 $\pm$ 0.047</u>	0.867 $\pm$ 0.002
GFN-SubTB	<u>0.620 $\pm$ 0.009</u>	0.686 $\pm$ 0.015	0.845 $\pm$ 0.039	0.845 $\pm$ 0.007	0.880 $\pm$ 0.008
GFN-FM	0.403 $\pm$ 0.218	<u>0.716 $\pm$ 0.071</u>	0.775 $\pm$ 0.041	0.926 $\pm$ 0.003	<u>0.922 $\pm$ 0.002</u>
Ours (LEAKGFN)	0.653 $\pm$ 0.068	0.760 $\pm$ 0.008	0.797 $\pm$ 0.070	0.926 $\pm$ 0.002	0.923 $\pm$ 0.003

Implementation. Molecules are built by iteratively attaching fragments from a vocabulary of 105 building blocks. Unless otherwise specified, we set the maximum trajectory length to eight fragments, and the shared encoder performs 8 NNConv-GRU message passing steps with 256 hidden dimensions, with a 3-layer MLP per head. See Appendix A for details.

4.2. Single-Objective Molecular Optimization

Table 2 presents results across five molecular optimization tasks. Methods are grouped by approach, with non-GFlowNet baselines at the top, GFlowNet variants with alternative training objectives in the middle, and flow-matching-based methods at the bottom. All main results in Table 2 use $\alpha = 1$.

Comparison with Non-GFlowNet Methods. All GFlowNet variants outperform the genetic algorithm (Graph-GA) and the reinforcement learning methods (REINVENT, GEGL) on most tasks, and the gap is largest on the drug-likeness metrics (QED, SA), where exploring multiple modes matters most.

Comparison with GFN-FM. Since LEAKGFN extends GFN-FM by adding the dual-head architecture, the bottom group of Table 2 enables direct comparison that isolates the contribution of flow decomposition. LEAKGFN matches or outperforms GFN-FM in HM score on all five tasks, though the gain is in aggregate rather than uniform across every individual metric.

The improvement is most pronounced on JNK3, where LEAKGFN raises the HM score from 0.403 to 0.653. Notably, GFN-FM exhibits high variance on this task with standard deviation of 0.218, indicating unstable training likely caused by the conflation of valid and forced terminals. By explicitly decomposing flow, LEAKGFN reduces this standard deviation from 0.218 to 0.068. Similar stabiliza-

tion is observed on GSK3 β , where the standard deviation decreases from 0.071 to 0.008, confirming that flow decomposition stabilizes training across challenging optimization landscapes. Even on tasks where GFN-FM already performs well (QED, SA), LEAKGFN maintains comparable performance and comparable variance. Detailed per-metric breakdowns in Appendix B reveal that the improvement primarily stems from stabilizing Uniqueness, which GFN-FM collapses on JNK3 (0.444 ± 0.468) and LEAKGFN recovers to 0.930 ± 0.063 , partly at the cost of lower Diversity on some tasks.

Comparison with Other GFlowNet Variants. Among GFlowNet methods, LEAKGFN achieves the highest HM on JNK3, GSK3 β , QED, and SA. On DRD2, GFN-DB achieves the best performance of 0.934, followed closely by GFN-TB at 0.933, while LEAKGFN obtains 0.797. At this budget the balance-based objectives are the stronger choice on DRD2. As Table 3 shows, however, the robustness advantage of LeakGFN becomes apparent when $L_{\max}$ is set conservatively larger.

Robustness to $L_{\max}$ Selection. A critical question for practitioners is how methods perform when the optimal trajectory length is unknown a priori. Table 3 examines this on DRD2 across all GFlowNet variants. At the well-tuned $L_{\max} = 8$, the balance-based objectives (GFN-DB, GFN-TB, GFN-SubTB) achieve excellent performance, with GFN-DB reaching 0.934. However, all balance-based methods exhibit *catastrophic sensitivity* to this hyperparameter. GFN-DB collapses from 0.934 at $L_{\max} = 8$ to 0.096 at $L_{\max} = 12$, GFN-TB from 0.933 to 0.106, and GFN-SubTB from 0.845 to 0.107. In contrast, LEAKGFN demonstrates *graceful degradation*, still reaching 0.435 at $L_{\max} = 12$, where the strongest balance-based baseline reaches only 0.107.

This robustness is crucial for real-world drug discovery, where the target molecule size distribution is often unknown

and practitioners prefer conservative, larger $L_{\max}$ values to avoid excluding potentially valuable candidates. Balance-based objectives (DB, TB, SubTB) inject the reward signal only at terminal states and satisfy it through trajectory- or transition-level balance constraints, making them sensitive to how much of the reachable state space lies near the truncation boundary, which grows with $L_{\max}$. Flow-matching-based methods (FM, LEAKGFN) instead enforce an in-flow/out-flow constraint at every state by explicitly enumerating parents, providing denser local supervision and offering inherent robustness. LEAKGFN further improves upon GFN-FM by explicitly modeling flow leakage, achieving the best performance at $L_{\max} \geq 10$. We report the full $\alpha \times L_{\max}$ grid in Appendix C.

Table 3. Robustness to $L_{\max}$ on DRD2. HM (mean) over 3 seeds.

Method	6	8	10	12
Ours (LEAKGFN)	0.865	0.797	0.517	0.435
GFN-FM	0.863	0.775	0.356	0.380
GFN-DB	0.926	0.934	0.136	0.096
GFN-TB	0.906	0.933	0.132	0.106
GFN-SubTB	0.927	0.845	0.158	0.107

4.3. Ablation Studies

We use JNK3 as the primary ablation task due to its challenging optimization landscape and sensitivity to truncation effects. All experiments use $L_{\max} = 8$ unless otherwise specified and are averaged over 3 random seeds. Table 4 presents ablation results removing key components of LEAKGFN. Replacing flow-guided exploration with the standard uniform ϵ -exploration of Eq. 19 scatters the sampler across the truncated space. Diversity rises from 0.54 to 0.64, while the success rate falls from 1.00 to 0.51 and uniqueness from 0.93 to 0.81, for a net HM change from 0.65 to 0.62. This confirms that biasing exploration toward chemically meaningful transitions improves reward quality by steering the sampler toward high-reward regions rather than spreading it thinly over the state space.

Removing the valid head entirely, which is equivalent to setting $\lambda = 1$, reduces the model to standard flow matching. Since $\pi_{\text{valid}} = \pi_{\text{chem}}$ there, the mixture of Eq. 20 degenerates and the setting falls back to uniform ϵ -exploration, reproducing GFN-FM of Table 2 on every metric. Performance degrades from 0.65 to 0.40 ($\Delta\text{HM} = 0.25$). Uniqueness drops from 0.93 to 0.44 and the success rate from 1.00 to 0.23, consistent with concentration on a small set of repeated, low-reward terminals. Comparing the two ablations, the dual-head decomposition accounts for the dominant improvement (+0.22 HM) independent of exploration strategy (+0.03 HM). This confirms that the architecture’s primary

contribution is improving *training dynamics* near the truncation boundary.

Table 4. Ablation study on JNK3. All metrics are averaged over 3 seeds. Success denotes the fraction of top-100 molecules with raw oracle score $q(x) \geq 0.5$. HM is computed within each seed before averaging and is therefore not the harmonic mean of the displayed means.

Method	HM	Score	Div	Uni	Success
LEAKGFN (full)	0.65	0.60	0.54	0.93	1.00
Uniform ϵ -expl.	0.62	0.51	0.64	0.81	0.51
w/o θ_{valid} ($\lambda = 1$)	0.40	0.40	0.64	0.44	0.23

Comparison with $R(x) = 0$ at Forced Terminals. A natural alternative to flow decomposition is to set $R(x) = 0$ at forced terminals, correcting the target support without architectural changes. We evaluate it on all five tasks under both objectives (Table 5, with the full grid in Table 14), and the outcome splits sharply between them. Under flow matching it degrades four of five tasks, severely on JNK3 (0.403 to 0.122) and DRD2 (0.775 to 0.181). Under trajectory balance it is close to neutral, losing on two tasks and gaining on three by small margins. This is what the mechanism predicts. Near-zero rewards at forced terminals propagate backward through *flow conservation*, suppressing flow near the boundary and causing *premature termination bias*. Trajectory balance constrains trajectory-level ratios instead, so the pathology is absent there and the shaping is at best marginally helpful. LEAKGFN avoids the failure mode by maintaining positive chemical flow F_{chem} at forced terminals while using λ to down-weight them in sampling. The one setting where it clearly exceeds LEAKGFN is DRD2 under trajectory balance, where balance-based objectives already outperform flow matching (Table 2).

Table 5. Comparison with $R(x) = 0$ reward shaping at forced terminals. HM score (mean $\pm$ std) over 3 seeds. The full five-task grid is in Table 14.

Method	JNK3	GSK3 β	DRD2
GFN-FM	0.403 $\pm$ 0.218	0.716 $\pm$ 0.071	0.775 $\pm$ 0.041
with $R(x) = 0$	0.122 $\pm$ 0.093	0.493 $\pm$ 0.163	0.181 $\pm$ 0.114
GFN-TB	0.624 $\pm$ 0.003	0.675 $\pm$ 0.008	0.933 $\pm$ 0.000
with $R(x) = 0$	0.491 $\pm$ 0.153	0.714 $\pm$ 0.011	0.924 $\pm$ 0.007
Ours (LEAKGFN)	0.653 $\pm$ 0.068	0.760 $\pm$ 0.008	0.797 $\pm$ 0.070

4.4. Analysis: Implicit Reachability Learning

A key claim of LEAKGFN is that λ learns a reachability-related gate without explicit labels. Appendix D shows λ decreasing with trajectory depth over training, from 0.93 at step 1 to 0.83 at step 8 by late training. This pattern emerges purely from flow matching without explicit reachability labels.

Table 6. Integration with TacoGFN on pocket-conditioned generation (CrossDocked benchmark). All methods, baselines included, were re-evaluated under the single protocol of Appendix E.

	Vina Dock($\downarrow$)		QED($\uparrow$)		SA($\uparrow$)		Div($\uparrow$)		Novel hit($\uparrow$)	
	Mean	Median	Mean	Median	Mean	Median	Mean	Median	Mean	Median
TargetDiff	-7.360	-7.556	0.487	0.490	0.602	0.585	0.743	0.735	0.079	0.055
Pocket2Mol	-7.486	-6.995	0.565	0.576	0.749	0.756	0.737	0.771	0.068	0.010
DecompDiff	-8.348	-8.247	0.368	0.346	0.557	0.565	0.600	0.598	0.030	0.000
TacoGFN	-8.237	-8.443	0.671	0.671	0.789	0.789	0.530	0.531	0.435	0.485
Ours	-8.589	-8.598	0.689	0.687	0.795	0.794	0.514	0.514	0.447	0.530

4.5. Generalization to Advanced Tasks

To demonstrate the generalizability of LEAKGFN, we evaluate on two advanced molecular generation scenarios, pocket-conditioned generation and multi-objective optimization.

Pocket-Conditioned Generation. We integrate LEAKGFN into TacoGFN (Shen et al., 2024), a state-of-the-art pocket-conditioned generation framework, replacing its single flow head with our two heads. Details are in Appendix E.

Table 6 presents results on the CrossDocked benchmark. Integrating LEAKGFN improves Vina Dock scores from -8.237 to -8.589 (mean), achieving the best binding affinity among all methods. Drug-likeness (QED), synthetic accessibility (SA), and novel hit rate also improve, while diversity is slightly lower (0.530 to 0.514).

Multi-Objective Generation. We evaluate LEAKGFN on multi-objective molecular optimization, where the goal is to generate diverse molecules that simultaneously satisfy multiple, potentially conflicting objectives. We use the GSK3 β + JNK3 dual-inhibition task, a challenging benchmark that requires balancing activity against two distinct kinase targets. We integrate LEAKGFN into the HyperNetwork-based GFlowNet framework (HN-GFN) (Zhu et al., 2023), replacing the single flow head with our dual-head architecture while preserving the preference conditioning mechanism. Performance is measured by hypervolume (HV) over the Pareto front and diversity (Div) among generated molecules, both defined in Appendix F.

LEAKGFN reaches a hypervolume of 0.673 against 0.550 for the previous best method, HN-GFN (Table 7), so the decomposition yields better Pareto coverage while keeping diversity comparable to the other GFlowNet methods.

5. Conclusion

We presented LEAKGFN, a dual-head architecture that addresses flow leakage in truncated GFlowNets. By decomposing flow into valid and leak components, our method

Table 7. Multi-objective generation on GSK3 β + JNK3.

Method	HV ($\uparrow$)	Div ($\uparrow$)
MOEA/D*	0.182 $\pm$ 0.045	n/a
NSGA-III*	0.364 $\pm$ 0.041	n/a
PS-GFN*	0.545 $\pm$ 0.055	0.786 $\pm$ 0.013
Concat-GFN*	0.534 $\pm$ 0.069	0.786 $\pm$ 0.004
FiLM-GFN*	0.431 $\pm$ 0.045	0.795 $\pm$ 0.014
HN-GFN*	0.550 $\pm$ 0.074	0.797 $\pm$ 0.015
Ours	0.673 $\pm$ 0.044	0.786 $\pm$ 0.008

*Results are taken from HN-GFN (Zhu et al., 2023).

shifts sampling toward accessible molecules while explicitly accounting for the mass that would exit the truncation boundary, and the valid head learns reachability through asymmetric terminal treatment without explicit supervision. Our analysis expresses the residual leakage in closed form and shows that, conditioned on producing a complete molecule, the sampler matches the target distribution. Experiments on five molecular optimization tasks show the best or tied-best HM on four (Table 2), and the modular design improves TacoGFN on pocket-conditioned generation and HN-GFN on multi-objective optimization.

Limitations. Like prior GFlowNet analyses, we characterize behavior at convergence and give no formal convergence guarantee, so the training-dynamics advantage rests on empirical evidence. The residual leakage of Eq. 24 is characterized rather than eliminated, since driving it to zero requires zero reward at forced terminals, which destabilizes training (Section 4.3). Finally, our oracles score incomplete fragments consistently low, and a surrogate that scored them highly would weaken the reward-side motivation, though λ is driven by the asymmetry of Table 1, not by reward magnitude. Promising directions include atom-level construction, continuous action spaces, and λ -driven adaptive truncation.

Acknowledgements

This research was supported by the National Research Foundation (NRF) funded by the Korean government (MSIT) (No. RS-2025-00523472).

Impact Statement

This paper presents work whose goal is to advance the field of molecular generation for drug discovery. Improved molecular generation methods may accelerate the identification of drug candidates, potentially reducing the time and cost of pharmaceutical development. While our method uses computational surrogates rather than wet-lab experiments, we note that any generated molecules would require extensive safety and efficacy testing before clinical use. The same capability could in principle be directed at harmful targets, and we regard release of generated structures without such screening as the main dual-use risk.

References

- Ahn, S., Kim, J., Lee, H., and Shin, J. Guiding deep molecular optimization with genetic exploration. *Advances in Neural Information Processing Systems*, 33:12008–12021, 2020.
- Alhossary, A., Handoko, S. D., Mu, Y., and Kwoh, C.-K. Fast, accurate, and reliable molecular docking with QuickVina 2. *Bioinformatics*, 31(13):2214–2216, 2015.
- Bengio, E., Jain, M., Korablyov, M., Precup, D., and Bengio, Y. Flow network based generative models for non-iterative diverse candidate generation. *Advances in Neural Information Processing Systems*, 34:27381–27394, 2021.
- Bengio, Y., Lahlou, S., Deleu, T., Hu, E. J., Tiwari, M., and Bengio, E. GFlowNet foundations. *Journal of Machine Learning Research*, 24(210):1–55, 2023.
- Bickerton, G. R., Paolini, G. V., Besnard, J., Muresan, S., and Hopkins, A. L. Quantifying the chemical beauty of drugs. *Nature Chemistry*, 4(2):90–98, 2012.
- Blaschke, T., Arús-Pous, J., Chen, H., Margreitter, C., Tyrchan, C., Engkvist, O., Papadopoulos, K., and Patronov, A. REINVENT 2.0: an AI tool for de novo drug design. *Journal of Chemical Information and Modeling*, 60(12):5918–5922, 2020.
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In Moschitti, A., Pang, B., and Daelemans, W. (eds.), *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1724–1734, Doha, Qatar, October 2014. Association for Computational Linguistics. doi: 10.3115/v1/D14-1179. URL <https://aclanthology.org/D14-1179/>.
- Ertl, P. and Schuffenhauer, A. Estimation of synthetic accessibility score of drug-like molecules based on molecular complexity and fragment contributions. *Journal of Cheminformatics*, 1(1):8, 2009.
- Fey, M. and Lenssen, J. E. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- Francoeur, P. G., Masuda, T., Sunseri, J., Jia, A., Iovanisci, R. B., Snyder, I., and Koes, D. R. Three-dimensional convolutional neural networks and a cross-docked data set for structure-based drug design. *Journal of Chemical Information and Modeling*, 60(9):4200–4215, 08 2020. ISSN 1549-9596. doi: 10.1021/acs.jcim.0c00411. URL <https://doi.org/10.1021/acs.jcim.0c00411>.
- Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., and Dahl, G. E. Neural message passing for quantum chemistry. In *International conference on machine learning*, pp. 1263–1272. PMLR, 2017.
- Gómez-Bombarelli, R., Wei, J. N., Duvenaud, D., Hernández-Lobato, J. M., Sánchez-Lengeling, B., Sheberla, D., Aguilera-Iparraguirre, J., Hirzel, T. D., Adams, R. P., and Aspuru-Guzik, A. Automatic chemical design using a data-driven continuous representation of molecules. *ACS Central Science*, 4(2):268–276, 2018.
- Guan, J., Qian, W. W., Peng, X., Su, Y., Peng, J., and Ma, J. 3D equivariant diffusion for target-aware molecule generation and affinity prediction. In *The Eleventh International Conference on Learning Representations*, 2023a. URL <https://openreview.net/forum?id=kJqXEPXMsE0>.
- Guan, J., Zhou, X., Yang, Y., Bao, Y., Peng, J., Ma, J., Liu, Q., Wang, L., and Gu, Q. DecompDiff: Diffusion models with decomposed priors for structure-based drug design. In Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., and Scarlett, J. (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 11827–11846. PMLR, 23–29 Jul 2023b. URL <https://proceedings.mlr.press/v202/guan23a.html>.
- Hains, A. W., Liang, Z., Woodhouse, M. A., and Gregg, B. A. Molecular semiconductors in organic photovoltaic cells. *Chemical Reviews*, 110(11):6689–6735, 2010.

- Hughes, J. P., Rees, S., Kalindjian, S. B., and Philpott, K. L. Principles of early drug discovery. *British Journal of Pharmacology*, 162(6):1239–1249, 2011.
- Jain, M., Bengio, E., Hernandez-Garcia, A., Rector-Brooks, J., Dossou, B. F., Ekbote, C. A., Fu, J., Zhang, T., Kilgour, M., Zhang, D., et al. Biological sequence design with GFlowNets. In *International conference on machine learning*, pp. 9786–9801. PMLR, 2022.
- Jain, M., Deleu, T., Hartford, J., Liu, C.-H., Hernandez-Garcia, A., and Bengio, Y. GFlowNets for AI-driven scientific discovery. *Digital Discovery*, 2(3):557–577, 2023.
- Jensen, J. H. A graph-based genetic algorithm and generative model/Monte Carlo tree search for the exploration of chemical space. *Chemical Science*, 10(12):3567–3572, 03 2019. ISSN 2041-6520. doi: 10.1039/c8sc05372c. URL <https://doi.org/10.1039/c8sc05372c>.
- Jiang, Y., Zhang, G., You, J., Zhang, H., Yao, R., Xie, H., Zhang, L., Xia, Z., Dai, M., Wu, Y., et al. PocketFlow is a data-and-knowledge-driven structure-based molecular generative model. *Nature Machine Intelligence*, 6(3): 326–337, 2024.
- Jin, W., Barzilay, R., and Jaakkola, T. Junction tree variational autoencoder for molecular graph generation. In *International conference on machine learning*, pp. 2323–2332. PMLR, 2018.
- Jin, W., Barzilay, R., and Jaakkola, T. Multi-objective molecule generation using interpretable substructures. In *International conference on machine learning*, pp. 4849–4859. PMLR, 2020.
- Kim, H., Kim, M., Choi, S., and Park, J. Genetic-guided GFlowNets for sample efficient molecular optimization. *Advances in Neural Information Processing Systems*, 37: 42618–42648, 2024a.
- Kim, M., Yun, T., Bengio, E., Zhang, D., Bengio, Y., Ahn, S., and Park, J. Local search GFlowNets. In *International Conference on Learning Representations*, 2024b.
- Madan, K., Rector-Brooks, J., Korablyov, M., Bengio, E., Jain, M., Nica, A. C., Bosc, T., Bengio, Y., and Malkin, N. Learning GFlowNets from partial episodes for improved convergence and stability. In *International conference on machine learning*, pp. 23467–23483. PMLR, 2023.
- Malkin, N., Jain, M., Bengio, E., Sun, C., and Bengio, Y. Trajectory balance: Improved credit assignment in GFlowNets. *Advances in Neural Information Processing Systems*, 35:5955–5967, 2022.
- Olivecrona, M., Blaschke, T., Engkvist, O., and Chen, H. Molecular de-novo design through deep reinforcement learning. *Journal of Cheminformatics*, 9(1):48, 2017.
- Pan, L., Malkin, N., Zhang, D., and Bengio, Y. Better training of GFlowNets with local credit and incomplete trajectories. In *International conference on machine learning*, pp. 26878–26890. PMLR, 2023.
- Peng, X., Luo, S., Guan, J., Xie, Q., Peng, J., and Ma, J. Pocket2Mol: Efficient molecular sampling based on 3D protein pockets. In *International conference on machine learning*, pp. 17644–17655. PMLR, 2022.
- Polishchuk, P. G., Madzhidov, T. I., and Varnek, A. Estimation of the size of drug-like chemical space based on GDB-17 data. *Journal of Computer-Aided Molecular Design*, 27(8):675–679, 2013.
- Rector-Brooks, J., Madan, K., Jain, M., Korablyov, M., Liu, C.-H., Chandar, S., Malkin, N., and Bengio, Y. Thompson sampling for improved exploration in GFlowNets. In *ICML 2023 Workshop on Structured Probabilistic Inference & Generative Modeling*, 2023. URL <https://openreview.net/forum?id=qdRQ5ejeAL>.
- Shen, T., Seo, S., Lee, G., Pandey, M., Smith, J. R., Cherkasov, A., Kim, W. Y., and Ester, M. TacoGFN: Target-conditioned GFlowNet for structure-based drug design. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=N8cPv95zOU>.
- Sun, J., Jeliaskova, N., Chupakhin, V., Golib-Dzib, J.-F., Engkvist, O., Carlsson, L., Wegner, J., Ceulemans, H., Georgiev, I., Jeliaskov, V., et al. ExCAPE-DB: an integrated large scale dataset facilitating Big Data analysis in chemogenomics. *Journal of Cheminformatics*, 9(1):17, 2017.
- Trott, O. and Olson, A. J. AutoDock Vina: improving the speed and accuracy of docking with a new scoring function, efficient optimization, and multithreading. *Journal of computational chemistry*, 31(2):455–461, 2010.
- Zhu, Y., Wu, J., Hu, C., Yan, J., Hsieh, C.-Y., Hou, T., and Wu, J. Sample-efficient multi-objective molecular optimization with GFlowNets. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=uoG1fLIK2s>.

A. Implementation Details

A.1. Model Architecture

Both the chemical head and valid head share a common state-action encoder θ_{emb} , implemented using the NNCONV message passing layer (Gilmer et al., 2017) from PyTorch Geometric (Fey & Lenssen, 2019) combined with a GRU (Cho et al., 2014) for iterative refinement. The architecture processes molecular graphs with the following specifications:

Graph Representation. Each molecule is represented as a graph where nodes correspond to molecular fragments (building blocks) and edges represent bonds between fragments. Node features are learned embeddings indexed by fragment type, and edge features are computed as the outer product of stem-type embeddings for the connected fragments.

Message Passing. The encoder performs $K = 8$ message passing steps. At each step k :

$$m_v^{(k)} = \text{LeakyReLU} \left(\text{NNConv}(h_v^{(k-1)}, \{h_u^{(k-1)}\}_{u \in \mathcal{N}(v)}, e_{uv}) \right) \quad (26)$$

$$h_v^{(k)} = \text{GRU}(m_v^{(k)}, h_v^{(k-1)}) \quad (27)$$

where $h_v^{(k)}$ is the hidden state of node v at step k , $\mathcal{N}(v)$ denotes the neighbors of v , and e_{uv} is the edge feature. The initial node embeddings $h_v^{(0)}$ are obtained by passing fragment embeddings through a 2-layer MLP with LeakyReLU activation.

Dual-Head Architecture. After message passing, the node representations are used to compute action logits:

Chemical Head (θ_{chem}): For each stem (attachment point) i at node v , the chemical head computes:

$$\log F_{\text{chem}}(s, a_i) = \text{MLP}_{\text{chem}}([h_v \| e_i^{\text{stem}}]) \quad (28)$$

where e_i^{stem} is the stem-type embedding and $\|$ denotes concatenation. The MLP consists of 3 layers: $\text{Linear}(2d \rightarrow d) \rightarrow \text{LeakyReLU} \rightarrow \text{Linear}(d \rightarrow d) \rightarrow \text{LeakyReLU} \rightarrow \text{Linear}(d \rightarrow |\mathcal{B}|)$, where $d = 256$ is the hidden dimension and $|\mathcal{B}| = 105$ is the number of building blocks.

Valid Head (θ_{valid}): The valid head has an identical architecture but outputs the valid fraction λ :

$$\lambda(s, a_i) = \sigma \left(\text{MLP}_{\text{valid}}([h_v \| e_i^{\text{stem}}]) \right) \quad (29)$$

where σ is the sigmoid function clamped below at 10^{-20} for numerical stability. This clamp is unrelated to the exploration rate ϵ of Section 3.5.

The final valid flow is computed as:

$$\log F_{\text{valid}}(s, a_i) = \log F_{\text{chem}}(s, a_i) + \log \lambda(s, a_i) \quad (30)$$

Stop Action: The stop action logit is computed from a global molecular representation using mean pooling:

$$\log F(s, \text{stop}) = \text{MLP}_{\text{stop}} \left(\frac{1}{|V|} \sum_{v \in V} h_v \right) \quad (31)$$

The stop MLP consists of 2 layers: $\text{Linear}(d \rightarrow d) \rightarrow \text{LeakyReLU} \rightarrow \text{Linear}(d \rightarrow 1)$.

A.2. Fragment Vocabulary

We use the vocabulary of 105 molecular fragments introduced in the original GFlowNet paper (Bengio et al., 2021). The vocabulary was curated from common drug-like building blocks and includes:

- Aromatic rings: benzene, pyridine, pyrimidine, indole, purine derivatives (20 fragments)
- Aliphatic rings: cyclohexane, piperidine, piperazine, morpholine (15 fragments)

- Functional groups: amides, carboxylic acids, sulfonamides, phosphates (25 fragments)
- Linkers and small groups: methyl, ethyl, hydroxyl, halogens, cyano (45 fragments)

Each fragment has predefined attachment points (stems) where new fragments can be connected. We use the same vocabulary file (`blocks_105.json`) as [Bengio et al. \(2021\)](#) to ensure fair comparison with prior work.

A.3. Auxiliary Stop Transitions

For every terminal state $s \in \mathcal{T}_{\text{valid}} \cup \mathcal{T}_{\text{forced}}$ encountered during trajectory sampling, we augment the training batch with the transition (s, stop, z_s) labeled with reward $R(s)$, where z_s is the absorbing terminal introduced in Section 3.4. This adds one transition per sampled trajectory, so a batch of 8 trajectories of average length 8 grows from 64 to 72 transitions, but ensures accurate stop action calibration without requiring a separate auxiliary loss. In the implementation z_s reuses the molecular features of s , so the transition is stored as (s, stop, s) with a terminal flag, where the repeated argument denotes feature reuse only, not a transition back into s . Both constraints of Section 3.4 are instances of $\mathcal{L}_{\text{valid}}^{\text{term}}$, and the auxiliary transitions are processed identically to regular ones during the forward pass.

A.4. Training Configuration

Table 8 summarizes the hyperparameters used for the single-objective flow matching experiments. The TacoGFN and HN-GFN integrations instead follow the settings of Appendices E and F.

Table 8. Hyperparameters for training.

Hyperparameter	Value
<i>Architecture</i>	
Hidden dimension (d)	256
Number of message passing steps (K)	8
Fragment vocabulary size ($ \mathcal{B} $)	105
<i>Optimization</i>	
Optimizer	Adam
Learning rate	5×10^{-4}
Weight decay	0
Gradient clipping	None
<i>Training</i>	
Batch size (trajectories)	8
Total iterations	30,000
Minimum blocks ($L_{\min}$)	2
Maximum blocks ($L_{\max}$)	8
<i>Exploration</i>	
Random action probability (ϵ)	0.05
<i>Loss Weights</i>	
Terminal loss coefficient (γ)	10
Chemical flow loss coefficient (α)	1.0
Log regularization constant	Task-specific

Reward Transformation. Raw oracle scores are transformed to rewards using:

$$R(x) = \left(\frac{q(x)}{\text{norm}} \right)^{\text{exp}} \quad (32)$$

where $q(x) \in [0, 1]$ denotes the raw oracle score of molecule x , and `norm` and `exp` are task-specific hyperparameters shown in Table 9. The same table lists the log regularization constant c , and every flow term appearing inside a logarithm in

Eqs. 16–18 is offset by c , so that $\log F$ is evaluated as $\log(F + c)$ on both sides of each ratio, while rewards are floored as $\log(R + 10^{-10})$. The reward $R(x)$ is used only for training, and all reported metrics are computed from the raw oracle score $q(x)$.

Table 9. Task-specific reward transformation parameters.

Task	norm	exp	log_reg c
JNK3	0.5	8	$(0.01/0.5)^8$
GSK3 β	0.2	12	$(0.01/0.2)^{12}$
DRD2	0.1	5	$(0.01/0.1)^5$
QED	0.8	6	$(0.01/0.8)^6$
SA	0.8	6	$(0.01/0.8)^6$

A.5. Oracle Models

Kinase Inhibition (JNK3, GSK3 β). We use Random Forest classifiers trained on bioactivity data from ExCAPE-DB (Sun et al., 2017). Input features are Morgan fingerprints with radius 2 and 1024 bits, computed using RDKit. The classifiers predict the probability of a molecule being active against the target kinase.

DRD2 Activity. We use a Support Vector Machine (SVM) classifier trained on DRD2 activity data. Input features are Morgan fingerprints with radius 3 and 2048 bits, using count-based features.

QED (Quantitative Estimate of Drug-likeness). Computed using the RDKit implementation of the QED score (Bickerton et al., 2012), which combines multiple molecular descriptors including molecular weight, logP, number of hydrogen bond donors/acceptors, polar surface area, number of rotatable bonds, and number of aromatic rings.

SA (Synthetic Accessibility). Computed using the RDKit implementation of the SA score (Ertl & Schuffenhauer, 2009). We normalize the raw SA score (originally in range [1, 10]) to [0, 1] using:

$$\text{SA}_{\text{normalized}} = \frac{10 - \text{SA}_{\text{raw}}}{9} \quad (33)$$

A.6. Evaluation Protocol

Sampling. For evaluation, we sample 1,000 molecules using the learned policy without exploration ($\epsilon = 0$). We select the top- k molecules ranked by oracle score $q(\cdot)$, with $k = 100$ for all experiments. Since the reward transformation is monotone in q , this ordering is identical to ranking by reward.

Metrics.

- **Score:** Mean *raw oracle score* of the top- k molecules: $\text{Score} = \frac{1}{k} \sum_{i=1}^k q(x_i)$ where $x_1, \dots, x_k$ are the top- k molecules ranked by q . Note that this is the untransformed oracle output, not the training reward $R(x)$ of Eq. 32.
- **Diversity (Div):** Average pairwise Tanimoto dissimilarity among top- k molecules: $\text{Div} = 1 - \frac{2}{k(k-1)} \sum_{i < j} \text{Tanimoto}(x_i, x_j)$ where Tanimoto similarity is computed on Morgan fingerprints (radius 3, 2048 bits).
- **Uniqueness (Uni):** Fraction of unique canonical SMILES strings among all 1,000 sampled molecules (computed before top- k selection).
- **Harmonic Mean (HM):** Primary evaluation metric combining Score, Diversity, and Uniqueness: $\text{HM} = \frac{3}{\frac{1}{\text{Score}} + \frac{1}{\text{Div}} + \frac{1}{\text{Uni}}}$

A.7. Computational Resources

All experiments were conducted on NVIDIA RTX 3090 GPUs (24 GB). Training a single model for 30,000 iterations takes approximately 2-3 hours depending on the oracle evaluation cost. Peak GPU memory usage is approximately 4 GB. We use 8 parallel sampling threads during training to improve throughput.

A.8. Baseline Implementations

For GFlowNet baselines (GFN-FM, GFN-TB, GFN-SubTB, GFN-DB), we use the same encoder, hidden dimensions, and training budget as LEAKGFN, differing only in the training objective and in the output heads that objective requires (a single flow head for GFN-FM, a learnable $\log Z$ for GFN-TB, and additional state-flow and backward-policy heads for GFN-DB). This ensures a fair comparison that isolates the contribution of our dual-head architecture.

- **GFN-FM:** Standard flow matching with single-head architecture.
- **GFN-TB:** Trajectory balance objective (Malkin et al., 2022) with log partition function $\log Z$ as a learnable parameter (initialized to 30, learning rate 5×10^{-3}).
- **GFN-SubTB:** Subtrajectory balance (Madan et al., 2023) with geometric subtrajectory weighting $\lambda_{\text{SubTB}} = 0.9$, distinct from the valid fraction λ of Section 3.3.
- **GFN-DB:** Detailed balance objective (Bengio et al., 2023), which enforces $F(s)P_F(s'|s) = F(s')P_B(s|s')$ at each transition, with the state flow $F(s)$ and the backward policy P_B parameterized by additional heads.

For non-GFlowNet baselines (Graph-GA, REINVENT, GEGL), we use results from the original papers or official implementations with default hyperparameters.

A.9. Computational Overhead

Table 10 compares the computational cost of LEAKGFN against the GFN-FM baseline. The dual-head architecture raises the parameter count from 1.01M to 1.23M, with the additional parameters coming primarily from the valid head MLP, while the shared encoder θ_{emb} keeps overhead modest. Crucially, inference time increases only from 51.1 ms to 52.5 ms, as both heads can be computed in parallel from the shared embeddings. This minimal overhead makes LEAKGFN practical for large-scale molecular screening campaigns where millions of candidates must be evaluated.

Table 10. Computational cost comparison between GFN-FM and LEAKGFN. Inference time is the wall-clock time to generate one complete molecule, i.e. a single trajectory of $L_{\text{max}} = 8$ fragment additions sampled with batch size 1, averaged over 1,000 rollouts on one RTX 3090. The measurement includes oracle evaluation, which is identical for both methods, so the relative overhead of the dual head alone is larger than the 2.8% shown here.

Method	#Parameters	Inference Time (ms)
GFN-FM	1.01M	51.1
LEAKGFN	1.23M (+22.2%)	52.5 (+2.8%)

Analysis. The modest parameter increase stems from the architectural design where both heads share the computationally expensive graph neural network encoder. Only the final MLP layers are duplicated, and the additional valid head MLP contributes approximately 220K parameters. The inference overhead is even smaller because the shared encoder dominates computation time and the two head computations can be parallelized on GPU.

Compared to the performance gains achieved by LEAKGFN (e.g., the HM score on JNK3 rises from 0.403 to 0.653, and the standard deviation on GSK3 β falls from 0.071 to 0.008), this computational overhead is negligible. The cost-effectiveness ratio strongly favors LEAKGFN for practical drug discovery applications where improved sample quality directly translates to reduced downstream experimental costs.

B. Detailed Experimental Results

This section provides detailed breakdowns of the main experimental results, showing individual metrics (Score, Diversity, Uniqueness) that constitute the harmonic mean (HM) reported in the main text. Tables 11, 12, and 13 present the detailed metrics corresponding to Table 2.

Analysis. The detailed results reveal that LEAKGFN’s improvement in HM score primarily stems from stabilizing Uniqueness. On JNK3, GFN-FM suffers from severe Uniqueness collapse (0.444 ± 0.468), indicating repeated sampling or mode collapse, consistent with (though not by itself proof of) concentration on forced terminals. LEAKGFN recovers Uniqueness to 0.930 ± 0.063 , with a far smaller standard deviation. This supports our hypothesis and is consistent with reduced concentration on forced terminals.

Interestingly, non-GFlowNet methods (Graph-GA, REINVENT, GEGL) often achieve higher Score but suffer from low Diversity and Uniqueness, reflecting mode collapse behavior. In contrast, GFlowNet variants (DB, TB, SubTB) maintain near-perfect Uniqueness (≥ 0.999) but with lower Score, demonstrating successful diversity-seeking behavior at the cost of reward optimization. LEAKGFN bridges this gap by maintaining high Uniqueness comparable to the balance-based methods while achieving competitive Score through flow decomposition.

Task dependence. The instability that flow decomposition removes is concentrated on a subset of tasks. On JNK3 the Uniqueness of GFN-FM has a standard deviation of 0.468 against a mean of 0.444, and on GSK3 β the deviation is 0.206. Under LEAKGFN these fall to 0.063 and 0.044. On DRD2 the two methods differ by 0.023 with small deviations on both sides, and on QED and SA both exceed 0.97 with deviations below 0.02. The tasks on which the decomposition helps are therefore those where flow matching is unstable to begin with, while on the remaining tasks both methods already sample almost entirely distinct molecules.

Table 11. Score comparison across tasks (corresponding to Table 2). All values are averaged over 3 random seeds.

Method	JNK3	GSK3 β	DRD2	QED	SA
Graph-GA	0.856 ± 0.018	0.723 ± 0.140	0.988 ± 0.008	0.931 ± 0.004	0.882 ± 0.003
REINVENT	0.773 ± 0.083	0.837 ± 0.062	1.000 ± 0.000	0.948 ± 0.000	0.887 ± 0.000
GEGL	0.685 ± 0.272	0.875 ± 0.041	1.000 ± 0.000	0.947 ± 0.000	0.892 ± 0.001
GFN-DB	0.397 ± 0.002	0.438 ± 0.004	0.944 ± 0.010	0.819 ± 0.103	0.726 ± 0.012
GFN-TB	0.398 ± 0.004	0.440 ± 0.013	0.943 ± 0.002	0.836 ± 0.120	0.734 ± 0.007
GFN-SubTB	0.395 ± 0.012	0.457 ± 0.022	0.717 ± 0.089	0.702 ± 0.014	0.765 ± 0.019
GFN-FM	0.399 ± 0.140	0.594 ± 0.029	0.661 ± 0.146	0.891 ± 0.007	0.909 ± 0.005
Ours	0.599 ± 0.071	0.626 ± 0.043	0.675 ± 0.150	0.904 ± 0.005	0.910 ± 0.011

Table 12. Diversity comparison across tasks (corresponding to Table 2). All values are averaged over 3 random seeds.

Method	JNK3	GSK3 β	DRD2	QED	SA
Graph-GA	0.443 ± 0.008	0.573 ± 0.073	0.717 ± 0.062	0.862 ± 0.008	0.811 ± 0.008
REINVENT	0.443 ± 0.142	0.484 ± 0.049	0.516 ± 0.039	0.750 ± 0.058	0.803 ± 0.005
GEGL	0.505 ± 0.171	0.470 ± 0.140	0.519 ± 0.081	0.835 ± 0.019	0.768 ± 0.002
GFN-DB	0.773 ± 0.002	0.852 ± 0.011	0.868 ± 0.002	0.898 ± 0.004	0.913 ± 0.005
GFN-TB	0.770 ± 0.003	0.853 ± 0.005	0.866 ± 0.002	0.887 ± 0.007	0.911 ± 0.006
GFN-SubTB	0.767 ± 0.009	0.846 ± 0.011	0.872 ± 0.003	0.889 ± 0.004	0.909 ± 0.005
GFN-FM	0.643 ± 0.116	0.807 ± 0.021	0.800 ± 0.052	0.899 ± 0.002	0.890 ± 0.012
Ours	0.541 ± 0.061	0.804 ± 0.014	0.826 ± 0.037	0.884 ± 0.004	0.888 ± 0.010

On QED and SA both objectives are already close to saturation, with all variants spanning 0.027 on QED and 0.058 on SA, so these two tasks carry little signal about the reward shaping, and the main text therefore reports the three tasks on which the effect is visible.

Table 13. Uniqueness comparison across tasks (corresponding to Table 2). All values are averaged over 3 random seeds.

Method	JNK3	GSK3 β	DRD2	QED	SA
Graph-GA	0.430 $\pm$ 0.020	0.458 $\pm$ 0.031	0.479 $\pm$ 0.023	0.501 $\pm$ 0.016	0.411 $\pm$ 0.009
REINVENT	0.608 $\pm$ 0.054	0.622 $\pm$ 0.087	0.378 $\pm$ 0.077	0.525 $\pm$ 0.137	0.315 $\pm$ 0.006
GEGL	0.793 $\pm$ 0.038	0.670 $\pm$ 0.171	0.777 $\pm$ 0.069	0.834 $\pm$ 0.018	0.493 $\pm$ 0.015
GFN-DB	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000
GFN-TB	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	1.000 $\pm$ 0.001	1.000 $\pm$ 0.001
GFN-SubTB	1.000 $\pm$ 0.001	1.000 $\pm$ 0.001	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	0.999 $\pm$ 0.000
GFN-FM	0.444 $\pm$ 0.468	0.814 $\pm$ 0.206	0.939 $\pm$ 0.027	0.996 $\pm$ 0.001	0.970 $\pm$ 0.009
Ours	0.930 $\pm$ 0.063	0.910 $\pm$ 0.044	0.962 $\pm$ 0.021	1.000 $\pm$ 0.000	0.975 $\pm$ 0.017

 Table 14. Full comparison with $R(x) = 0$ reward shaping at forced terminals across all five tasks and both training objectives (corresponding to Table 5). HM score (mean $\pm$ std) over 3 random seeds.

Method	JNK3	GSK3 β	DRD2	QED	SA
GFN-FM	0.403 $\pm$ 0.218	0.716 $\pm$ 0.071	0.775 $\pm$ 0.041	0.926 $\pm$ 0.003	0.922 $\pm$ 0.002
with $R(x) = 0$	0.122 $\pm$ 0.093	0.493 $\pm$ 0.163	0.181 $\pm$ 0.114	0.920 $\pm$ 0.001	0.925 $\pm$ 0.002
GFN-TB	0.624 $\pm$ 0.003	0.675 $\pm$ 0.008	0.933 $\pm$ 0.000	0.899 $\pm$ 0.047	0.867 $\pm$ 0.002
with $R(x) = 0$	0.491 $\pm$ 0.153	0.714 $\pm$ 0.011	0.924 $\pm$ 0.007	0.914 $\pm$ 0.002	0.893 $\pm$ 0.005
Ours (LEAKGFN)	0.653 $\pm$ 0.068	0.760 $\pm$ 0.008	0.797 $\pm$ 0.070	0.926 $\pm$ 0.002	0.923 $\pm$ 0.003

C. Sensitivity Analysis on α

We investigate the interaction between the loss weight α and the maximum trajectory length $L_{\max}$ on DRD2. This analysis tests our hypothesis that the chemical head becomes more important as truncation effects intensify.

 Table 15. Interaction between α and $L_{\max}$ on DRD2. HM score (mean) over 3 random seeds. Higher is better.

	$L_{\max} = 6$	$L_{\max} = 8$	$L_{\max} = 10$	$L_{\max} = 12$
GFN-FM	0.863 $\pm$ 0.038	0.775 $\pm$ 0.041	0.356 $\pm$ 0.151	0.380 $\pm$ 0.241
$\alpha = 0$	0.825 $\pm$ 0.044	0.801 $\pm$ 0.035	0.591 $\pm$ 0.086	0.350 $\pm$ 0.037
$\alpha = 0.1$	0.877 $\pm$ 0.025	0.829 $\pm$ 0.122	0.574 $\pm$ 0.115	0.528 $\pm$ 0.068
$\alpha = 0.5$	0.880 $\pm$ 0.013	0.780 $\pm$ 0.054	0.539 $\pm$ 0.103	0.355 $\pm$ 0.052
$\alpha = 1.0$	0.865 $\pm$ 0.022	0.797 $\pm$ 0.070	0.517 $\pm$ 0.086	0.435 $\pm$ 0.078
$\alpha = 2.0$	0.895 $\pm$ 0.013	0.260 $\pm$ 0.402	0.344 $\pm$ 0.127	0.466 $\pm$ 0.072

Effect of α . When $\alpha = 0$, the chemical head receives no direct supervision and is trained only through gradients backpropagated from $\mathcal{L}_{\text{valid}}$. Even $\alpha = 0$ improves substantially over GFN-FM at $L_{\max} = 10$ (0.591 vs 0.356), demonstrating that the dual-head architecture itself contributes to stability regardless of explicit chemical flow supervision. At $L_{\max} = 12$, however, it falls slightly below GFN-FM (0.350 vs 0.380), so explicit supervision still matters at the largest budget.

However, excessively large α can destabilize training. At $\alpha = 2.0$ with $L_{\max} = 8$, we observe a significant performance drop (0.260 $\pm$ 0.402). This instability arises from *conflicting gradient signals* between the two loss terms. $\mathcal{L}_{\text{valid}}$ encourages $F_{\text{valid}}(s, a) \rightarrow 0$ at forced terminals, while $\mathcal{L}_{\text{chem}}$ maintains $F_{\text{chem}}(s, a) > 0$. When α is large, the chemical loss dominates, disrupting the delicate balance required for λ to learn correct reachability estimates.

Interaction with $L_{\max}$. At small $L_{\max}$ (e.g., 6), all configurations perform similarly well, as truncation effects are minimal. As $L_{\max}$ increases, the gap between LeakGFN variants and GFN-FM widens substantially. Notably, $\alpha = 2.0$ exhibits instability at $L_{\max} = 8$ (0.260 $\pm$ 0.402), suggesting that overweighting the chemical loss can destabilize training when valid and chemical flows diverge significantly. Moderate values ($\alpha \in [0.1, 1.0]$) provide the best balance between

leveraging chemical flow information and maintaining stable optimization.

Gradient Flow Analysis. The multiplicative decomposition $F_{\text{valid}} = F_{\text{chem}} \cdot \lambda$ might appear to starve F_{chem} of gradient when $\lambda \rightarrow 0$, since $\partial F_{\text{valid}} / \partial F_{\text{chem}} = \lambda$. This concern does not apply to our parameterization. Both heads emit log-scale flow and the losses of Eqs. 16–18 are squared log-ratios, so $\log F_{\text{valid}} = \log F_{\text{chem}} + \log \lambda$ and the gradient of $\mathcal{L}_{\text{valid}}$ with respect to $\log F_{\text{chem}}$ does not depend on λ . Small λ therefore leaves learning of the chemical head intact, and the same holds for gradients reaching the shared encoder θ_{emb} . What $\alpha > 0$ contributes is an independent conservation signal on F_{chem} at states where $\mathcal{L}_{\text{valid}}$ alone constrains only the product, rather than a rescue from starvation.

Empirically, we observe stable training across $\alpha \in [0, 1]$ in Table 15. The instability at $\alpha = 2.0$ arises from *conflicting gradient signals* ($\mathcal{L}_{\text{valid}}$ pushing $F_{\text{valid}}(s, a) \rightarrow 0$ at forced terminals while $\mathcal{L}_{\text{chem}}$ maintains $F_{\text{chem}}(s, a) > 0$) rather than from gradient starvation.

Key Finding: Architecture vs. Supervision. A notable finding is that $\alpha = 0$ (no explicit chemical supervision) can be competitive at larger L_{max} . It is the strongest setting at $L_{\text{max}} = 10$ (0.591), although it is also the weakest at $L_{\text{max}} = 12$ (0.350), so the effect varies with the truncation budget. This suggests that the *architectural* contribution of the dual-head decomposition, which enables implicit reachability learning through λ , is more fundamental than the explicit chemical flow supervision.

The chemical head serves primarily as a “base flow” that λ modulates, and this modulation can be learned effectively through gradients from the valid loss alone.

Variance Reduction. At the challenging settings $L_{\text{max}} \geq 10$, every LEAKGFN variant has a smaller standard deviation than GFN-FM (e.g., 0.086 vs 0.151 at $L_{\text{max}} = 10$, and 0.037–0.078 vs 0.241 at $L_{\text{max}} = 12$). At $L_{\text{max}} \leq 8$ the two are comparable for $\alpha \leq 1$, while $\alpha = 2$ is clearly unstable at $L_{\text{max}} = 8$. Even $\alpha = 0$ achieves the reduction at large L_{max} , indicating that the dual-head architecture provides implicit regularization independent of the chemical loss weight.

Practical Recommendation. Based on this analysis, we provide the following guidelines for practitioners:

- Use $\alpha \in [0.1, 0.5]$ when L_{max} is set close to the expected molecule size distribution.
- Do not treat $\alpha = 0$ as a safe default at large L_{max} . It is the best setting at $L_{\text{max}} = 10$ but the weakest at $L_{\text{max}} = 12$ (0.350, below GFN-FM’s 0.380). Search $\alpha \in [0.1, 1]$ instead.
- Avoid $\alpha \geq 2.0$ due to potential training instability, especially at moderate L_{max} values.
- When in doubt, $\alpha = 1$ provides a reasonable default with consistent performance across most settings.

D. Implicit Reachability Learning Analysis

Figure 2 visualizes the learned valid fraction across trajectory steps during training, providing qualitative evidence consistent with implicit reachability learning.

We fix a set of pre-sampled length-8 trajectories and measure λ at each step across training checkpoints. Because the trajectories are held fixed, the reported trend reflects learned reachability rather than a shift in the sampling distribution. Early in training (0-10K), λ incorrectly increases with trajectory step due to random initialization. During mid training (10K-20K), λ becomes flat as the model transitions. By late training (20K-30K), λ exhibits the correct decreasing trend, dropping from 0.93 at step 1 to 0.83 at step 8. The steepest drop falls between the second and third steps, which aligns with scaffold determination in fragment-based generation. This pattern emerges purely from flow matching without explicit reachability labels. Each point is the mean λ over all add actions available at that depth, so the curve stays above zero even where the actions leading past the boundary are driven toward it. The figure therefore shows a depth-related reachability trend rather than a calibrated estimate of completion probability.

E. Pocket-Conditioned Generation Setup

This section provides detailed experimental setup for the pocket-conditioned generation experiments in Section 4.5.

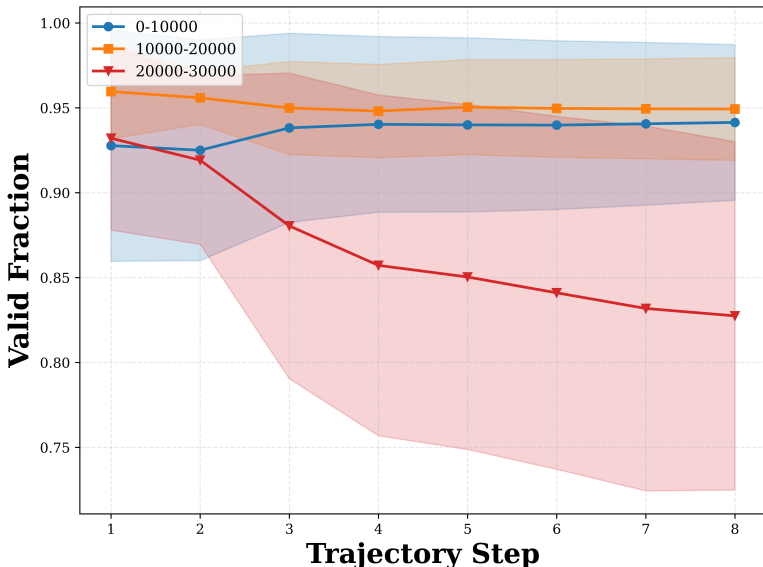


Figure 2. Evolution of learned valid fraction λ during training on JNK3 ($L_{\max} = 8$). Curves show the mean λ over checkpoints within each training stage, and shaded regions denote one standard deviation across those checkpoints. The model progressively learns to associate later trajectory steps with lower reachability.

Setup. TacoGFN generates molecules conditioned on protein binding pockets by combining a pocket encoder with fragment-based GFlowNet generation trained with the trajectory balance objective (Malkin et al., 2022), and this integration therefore also shows that the decomposition transfers beyond the flow matching objective used in Section 4. Like other fragment-based approaches, TacoGFN requires trajectory truncation, making it susceptible to flow leakage. Our dual-head architecture integrates as a plug-and-play module. We replace TacoGFN’s single flow prediction head with our chemical and valid heads while keeping the pocket encoder and fragment embeddings unchanged. This requires minimal architectural modification.

Integration with the trajectory balance objective. TacoGFN parameterizes the forward policy by per-action-type logit heads on top of a shared graph transformer. We keep that backbone and duplicate only the logit heads, with one bank producing $\log F_{\text{chem}}(s, a)$ and a second producing $\lambda(s, a)$, so that

$$\log F_{\text{valid}}(s, a) = \log F_{\text{chem}}(s, a) + \log \lambda(s, a), \quad (34)$$

with λ the clamped sigmoid of Appendix A. Unlike the flow matching setting, every action type including `stop` carries a λ head, since trajectory balance imposes no separate $F(s, \text{stop}) = R(s)$ constraint to calibrate. Because trajectory balance constrains trajectory-level ratios rather than state-level in-flow and out-flow, the asymmetric terminal treatment of Section 3.4 takes a different form here. Each head is trained by its own trajectory balance loss, and the two are optimized in alternation at every iteration, each on its own batch of sampled trajectories, one scoring its batch under $\log F_{\text{valid}}$ and the other under $\log F_{\text{chem}}$. The second loss is what keeps F_{chem} supervised independently of λ , playing the role $\mathcal{L}_{\text{chem}}$ plays under flow matching, and the two receive equal weight, corresponding to $\alpha = 1$. Only the decomposition transfers. The flow-guided exploration of Section 3.5 is not used here, and we retain TacoGFN’s defaults of ϵ -random actions with probability 0.01 and a truncation budget of $L_{\max} = 9$.

Baselines. We compare against state-of-the-art structure-based drug design methods:

- **Diffusion-based approaches:** TargetDiff (Guan et al., 2023a) and DecompDiff (Guan et al., 2023b)
- **Autoregressive methods:** Pocket2Mol (P2M) (Peng et al., 2022)
- **GFlowNet baseline:** The original TacoGFN without flow decomposition

Metrics. We download the released molecules for every baseline and re-score all methods with a single evaluation pipeline, so that the docking setup, fingerprint definition, and metric thresholds are identical across rows of Table 6. The resulting values therefore differ from those printed in the original papers, which use different filters and docking settings. We evaluate on the CrossDocked benchmark (Francoeur et al., 2020) using five metrics:

- **Vina Dock:** Predicted binding affinity (kcal/mol). All molecules are re-docked with QuickVina 2.1 (Alhossary et al., 2015), an accelerated reimplementation of AutoDock Vina (Trott & Olson, 2010), using the same box and exhaustiveness settings for every method. Lower values indicate stronger predicted binding.
- **QED:** Quantitative estimate of drug-likeness (Bickerton et al., 2012), ranging from 0 to 1.
- **SA:** Synthetic accessibility score, normalized to $[0, 1]$ where higher is more synthetically accessible.
- **Diversity:** Average pairwise Tanimoto dissimilarity (on Morgan fingerprints, radius 2, 2048 bits) among generated molecules for each pocket.
- **Novel hit rate:** Fraction of generated molecules satisfying (i) Vina Dock < -8.0 kcal/mol, and (ii) maximum Tanimoto similarity to any CrossDocked training molecule < 0.4 . This metric measures the ability to discover novel, high-affinity binders. Note that this definition is deliberately simpler than the “novel hit” of Shen et al. (2024), which additionally requires QED > 0.5 and SA > 0.5 and uses a similarity threshold of 0.5, so our values are not directly comparable to those reported there.

F. Multi-Objective Generation Setup

This section provides detailed experimental setup for the multi-objective generation experiments in Section 4.5.

Task. We evaluate on the GSK3 β + JNK3 dual-objective task, where the goal is to generate molecules that simultaneously inhibit both kinases. This is a challenging multi-objective optimization problem as the two objectives may have conflicting structural requirements.

Integration with HN-GFN. We adopt the HyperNetwork-based preference conditioning framework of HN-GFN (Zhu et al., 2023) and integrate our dual-head architecture as a drop-in replacement for the flow prediction module. Unlike the TacoGFN integration of Appendix E, HN-GFN is trained with flow matching over enumerated parents, so the objective of Section 3.4 transfers unchanged, including the asymmetric terminal treatment and the auxiliary stop transition. Specifically:

- **Flow decomposition:** A second stem head outputs $\lambda(s, a)$ as a sigmoid clamped to $[10^{-20}, 1]$, and the valid flow is formed additively in log space, $\log F_{\text{valid}}(s, a) = \log F_{\text{chem}}(s, a) + \log \lambda(s, a)$, exactly as in Appendix A. As in Section 3, the decomposition applies to fragment-addition actions only, and the stop output remains a single undecomposed head.
- **Preference conditioning:** A preference vector $\omega \in \Delta^{K-1}$ (where $K = 2$ for dual objectives) is sampled uniformly during training. The hypernetwork generates weights for both the chemical head and valid head conditioned on ω , so the decomposition is applied independently for each preference and λ can capture reachability patterns that vary across regions of the Pareto front.
- **Reward scalarization:** Following HN-GFN, we use the weighted sum $R(x; \omega) = \sum_{k=1}^K \omega_k R_k(x)$ where $R_k(x)$ is the reward for objective k .

We use $L_{\text{max}} = 8$, matching the single-objective experiments of Section 4. All other hyperparameters (learning rate, batch size, network architecture) follow the original HN-GFN implementation to ensure fair comparison.

Baselines. We compare against both evolutionary and GFlowNet-based multi-objective methods:

- **Evolutionary algorithms:** MOEA/D and NSGA-III, classical multi-objective optimization methods
- **GFlowNet variants:**

- PS-GFN: Preference-conditioned GFlowNet trained separately per preference, an upper reference for per-preference specialization
- Concat-GFN: Concatenates preference vector to state representation
- FiLM-GFN: Uses Feature-wise Linear Modulation for preference conditioning
- HN-GFN: HyperNetwork-based preference conditioning (Zhu et al., 2023)

Both metrics are computed with the released HN-GFN evaluation code without modification, and the baseline values in Table 7 are taken directly from Zhu et al. (2023), so the comparison uses a single evaluation protocol throughout.

Metrics. We evaluate multi-objective performance using the following metrics:

- **Hypervolume (HV):** Volume of the region of objective space that is dominated by the Pareto front and bounded below by the reference point $r = (0, 0)$. Higher values indicate better coverage and quality of the Pareto front. Formally,

$$\text{HV} = \text{Vol}(\{y \in \mathbb{R}^K : \exists x \in \mathcal{P}, r \preceq y \preceq f(x)\}),$$

where $\mathcal{P}$ is the Pareto front, $f(x) = (q_1(x), q_2(x))$ collects the per-objective oracle scores, and $\preceq$ denotes component-wise inequality. Including the lower bound $r \preceq y$ is what makes the dominated region bounded, and hence the volume finite.

- **Diversity (Div):** Average pairwise Tanimoto dissimilarity on Morgan fingerprints among the generated molecules. This measures structural diversity of the discovered trade-off solutions.

G. Generated Molecules

We present representative molecules generated by LEAKGFN on each optimization task. For each task, we show the top-scoring molecules along with their oracle scores.

G.1. JNK3 Kinase Inhibition

Figure 3 shows the top-9 molecules generated by LEAKGFN for JNK3 kinase inhibition. These molecules exhibit structural features commonly associated with kinase inhibitors, including aromatic rings and hydrogen bond acceptors.

G.2. GSK3 β Kinase Inhibition

Figure 4 shows the top-9 molecules generated for GSK3 β inhibition.

G.3. DRD2 Activity

Figure 5 shows the top-9 molecules generated for DRD2 receptor activity.

G.4. Drug-likeness (QED)

Figure 6 shows molecules optimized for drug-likeness (QED score).

G.5. Synthetic Accessibility (SA)

Figure 7 shows molecules optimized for synthetic accessibility (SA score).

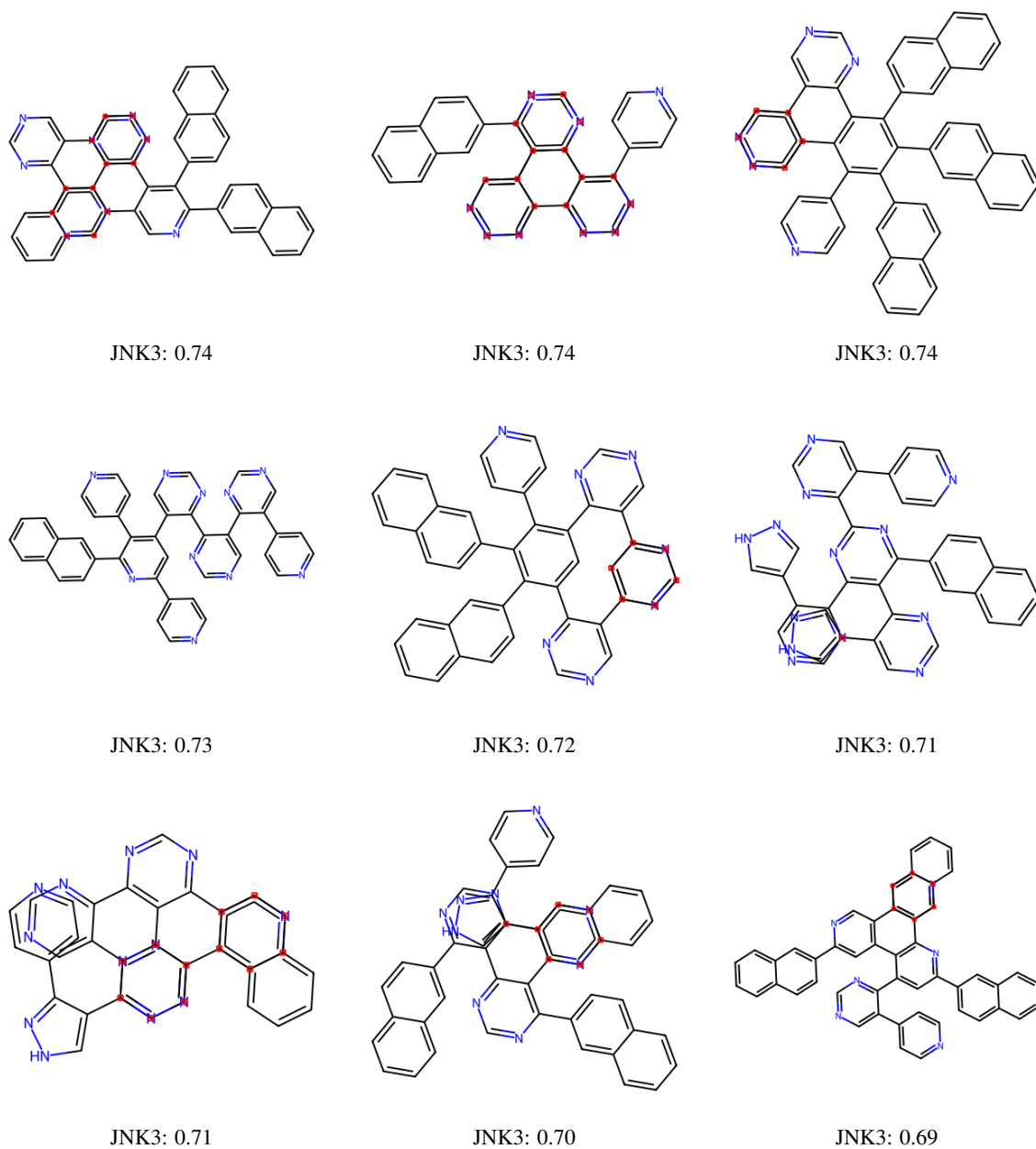


Figure 3. Top-9 molecules generated by LeakGFN for JNK3 kinase inhibition. Each molecule is shown with its predicted activity score.

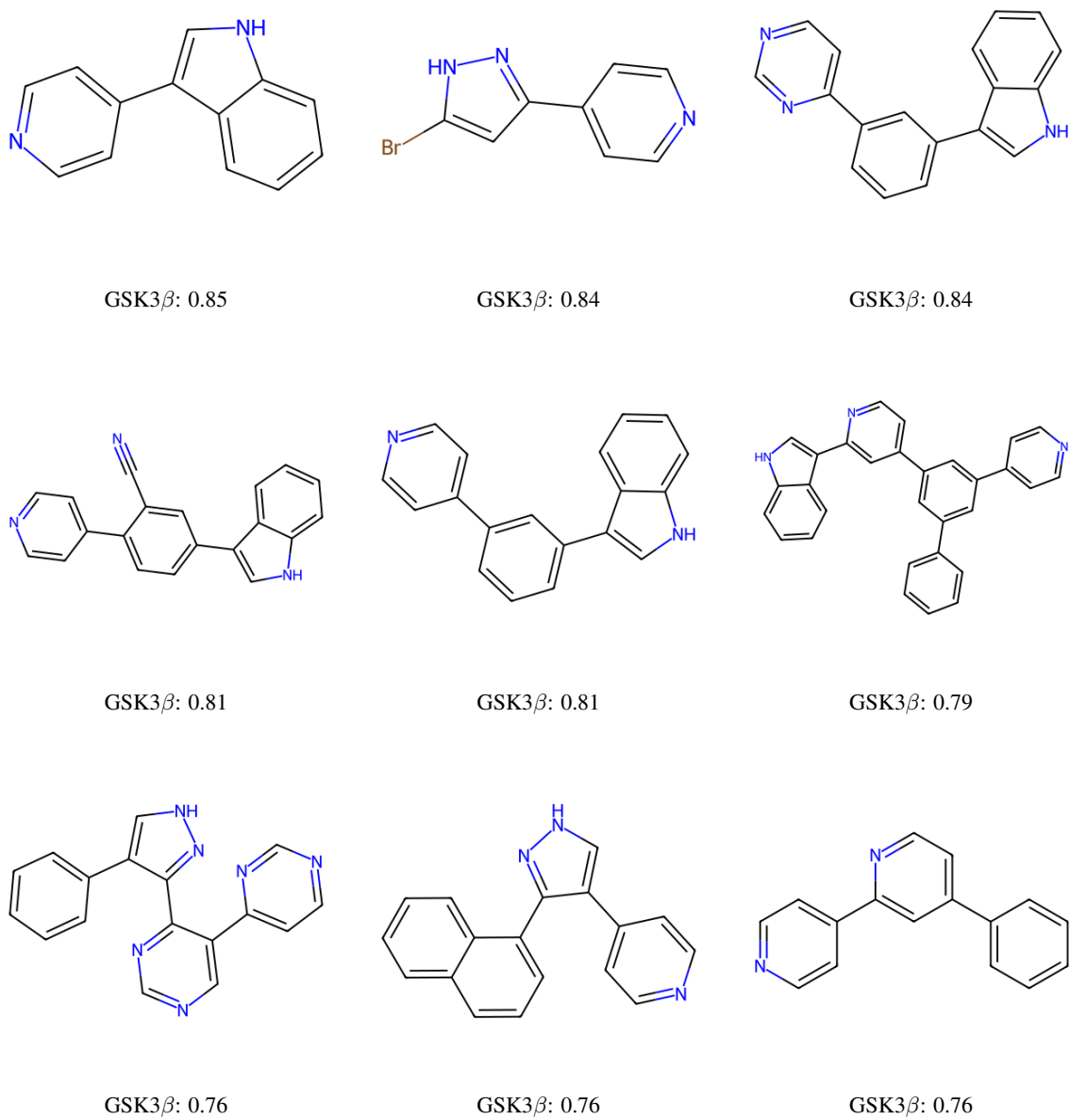
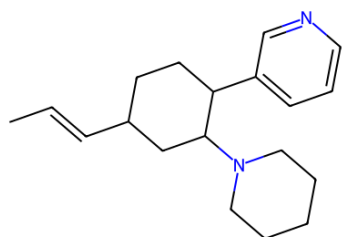
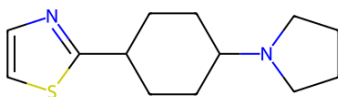


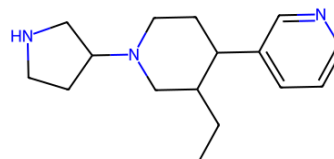
Figure 4. Top-9 molecules generated by LEAKGFN for GSK3 β kinase inhibition.



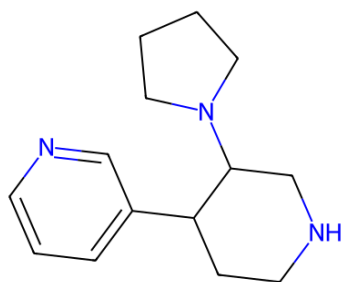
DRD2: 0.895



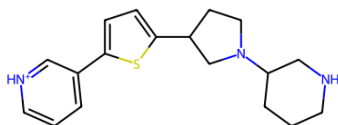
DRD2: 0.885



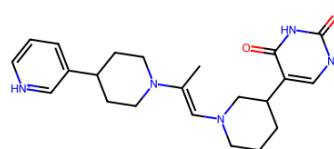
DRD2: 0.861



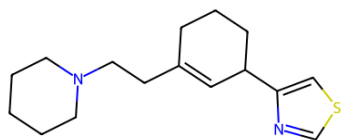
DRD2: 0.856



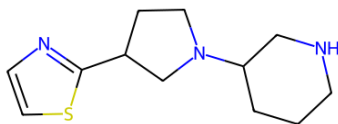
DRD2: 0.838



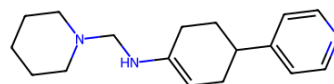
DRD2: 0.833



DRD2: 0.803



DRD2: 0.771



DRD2: 0.771

Figure 5. Top-9 molecules generated by LEAKGFN for DRD2 receptor activity.

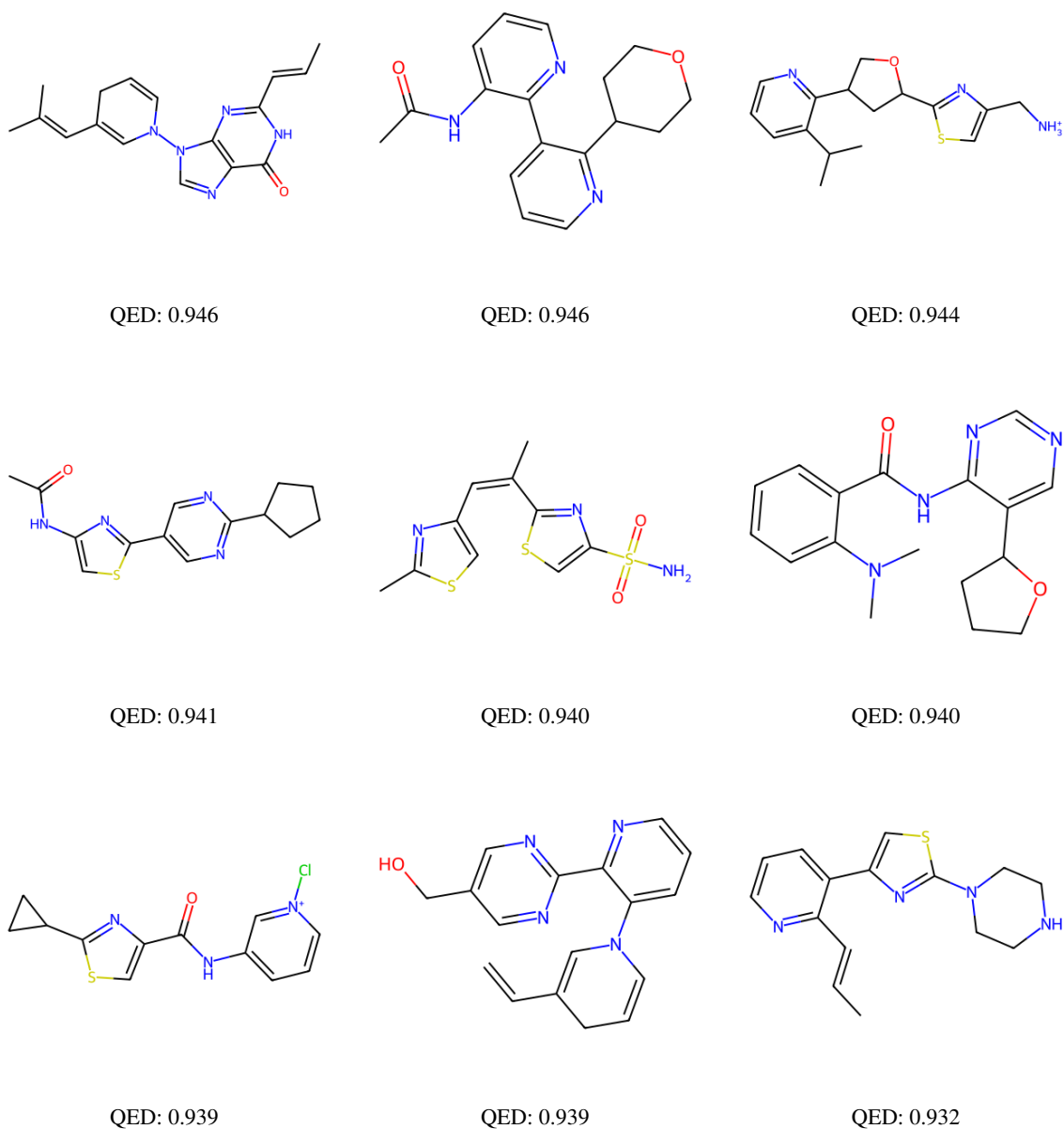


Figure 6. Top-9 molecules generated by LEAKGFN for drug-likeness optimization.

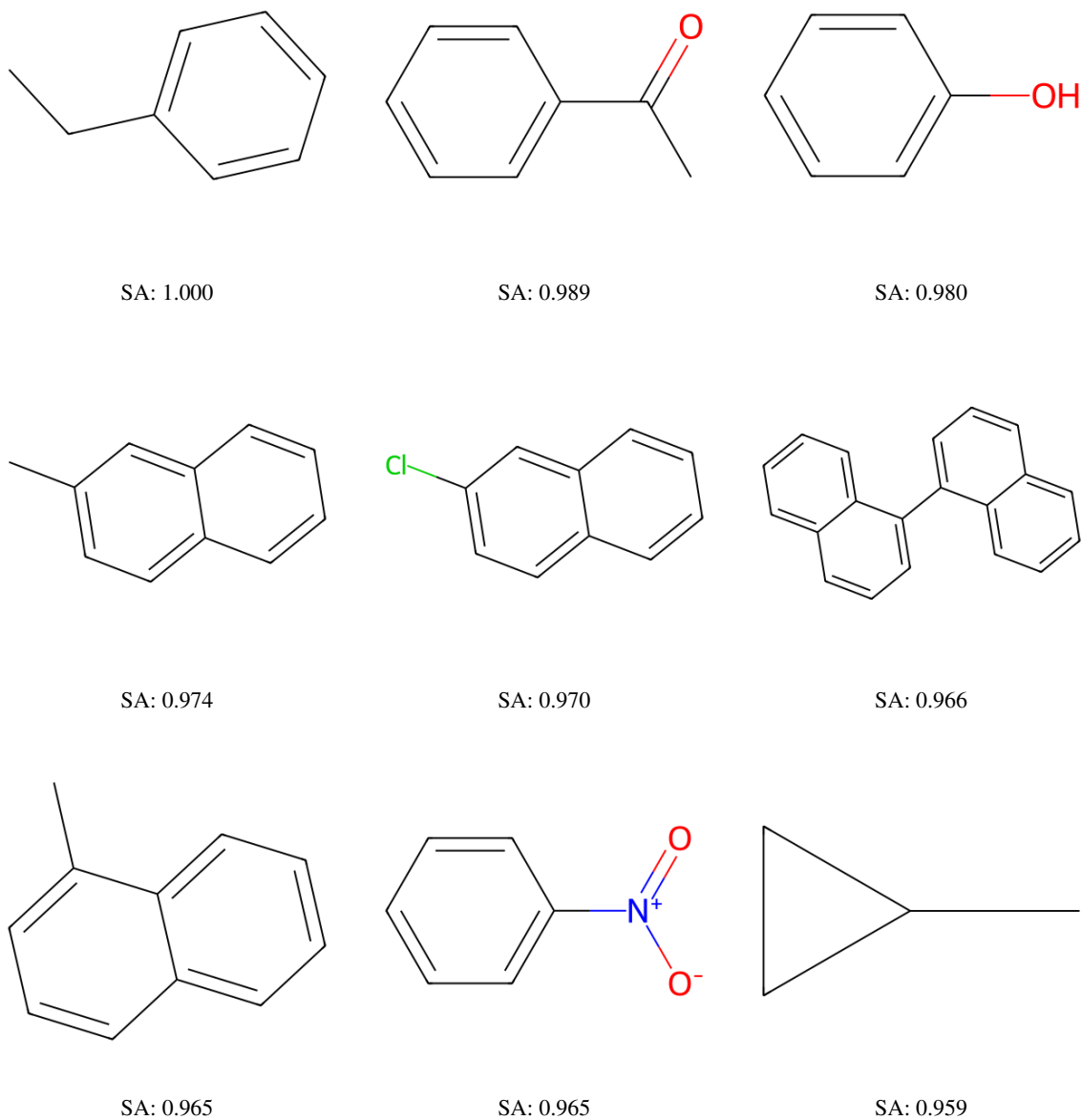


Figure 7. Top-9 molecules generated by LEAKGFN for synthetic accessibility optimization.